UC Berkeley CS162 操作系统与系统编程（Operating Systems and System Programming）学习笔记

> **课程**：UC Berkeley CS 162 *Operating Systems and System Programming*，Fall 2026，4 学分。 **讲师**：John Kubiatowicz（课程主页 Instructor 一栏；办公室 Gateway 1110A）。 **课程主页**：https://cs162.org/ ｜ 资源页：https://cs162.org/resources/ ｜ 政策页：https://cs162.org/policies/ **公开讲义**：`https://cs162.org/static/lectures/N.pdf`。本笔记实际下载并逐讲提取文本的是 **Lecture 1–19**（N = 1…19 全部返回 HTTP 200）；Lecture 20–28 的 PDF 在本学期站点上**返回 404**，相关章节只能依据主页日程表的讲题文字、教材章节号与公开教材原理撰写，并在正文中明确标注。 **教材**：Anderson & Dahlin《Operating Systems: Principles and Practice》(2nd ed.)，课程中以缩写 **A&D** 引用；免费替代教材 OSTEP（*Operating Systems: Three Easy Pieces*）https://pages.cs.wisc.edu/~remzi/OSTEP/ 。 **项目平台**：Pintos 教学操作系统；Pintos 文档 https://cs162.org/static/proj/pintos-docs/ 。 **访问日期**：所有 URL 均于 2026-11 访问。 **版权声明**：本文档为个人学习笔记，课程材料（讲义、项目说明、文档）版权归 UC Berkeley 与课程作者所有，仅用于个人学习目的。凡材料未明说、由笔者推断之处均标注“推断”；凡课程当学期可能变动之处均标注“以课程当学期主页为准”。

1.1 一句话定位
1.2 为什么这门课难，且难在哪
1.3 先修要求（政策页原文归纳）
1.4 评分与考试（以政策页为准）
1.5 日程骨架（来自主页 Schedule 表）
1.6 四个项目 + 六个作业总览
1.7 与 CS111 / CSAPP / CS144 的定位差异
2.1 动机：为什么需要一层“中间人”
2.2 三种角色：Referee / Illusionist / Glue
2.3 抽象映射表（背下来）
2.4 双模式操作：保护的全部基础
2.5 复杂度论证：为什么 OS 必须“工程化”
2.6 历史与技术趋势（考点的“背景板”）
2.7 关键数据结构与不变量（本章层面）
2.8 经典陷阱
3.1 动机：把“魔法”拆成四个可实现的部件
3.2 线程：一个“虚拟核”
3.3 地址空间与两种“翻译”方案
3.3.1 无翻译的多道程序（Loader/Linker 重定位）
3.3.2 Base & Bound（基址+界限）
3.3.3 分段（Segmentation）
3.3.4 分页（Paging）
3.4 进程：为什么需要它
3.5 三种“进入内核”的方式（必背）
3.6 内核栈与用户栈：双栈模型
3.7 关键数据结构与不变量
3.8 经典陷阱
4.1 动机：把“一个核”复用成“很多个核”
4.2 线程的三状态模型与生命周期
4.3 进程创建：fork / exec / wait 的语义
4.4 地址空间布局（x86 32 位，PintOS）
4.5 上下文切换：到底保存了什么
4.6 用户级线程 vs 内核级线程
4.7 关键数据结构与不变量
4.8 经典陷阱
5.1 动机：进程之间如何“说话”

5.2 “一切皆文件”（Everything is a File）
5.3 文件的两层抽象：fd 与 FILE*
5.4 dup / pipe / 重定向：shell 的实现基础
5.5 阻塞语义与调度器的交互
5.6 关键数据结构与不变量
5.7 经典陷阱
6.1 动机：跨机器的“文件 I/O”
6.2 Socket 与连接的表示
6.3 服务器并发模型
6.4 IPC 全景
6.5 关键数据结构与不变量
6.6 经典陷阱
7.1 动机：并发是“送命题”，也是“送分题”
7.2 原子性：并发推理的地基
7.3 临界区与互斥：术语精确化
7.4 Too Much Milk：为什么“只靠读写”不行
7.5 生产者—消费者与有界缓冲区
7.6 关键数据结构与不变量
7.7 经典陷阱
8.1 动机：从“纸面算法”到“内核原语”
8.2 单核方案：关中断
8.3 多核方案：原子读改写指令
8.4 两态锁与三态锁（futex 的核心思想）
8.5 关键数据结构与不变量（PintOS 对应实现）
8.6 经典陷阱
9.1 动机：一把锁不够用
9.2 定义与语义
9.3 两种典型用法
9.4 顺序为什么重要（死锁的第一课）
9.5 用信号量实现其它原语
9.6 关键数据结构与不变量
9.7 经典陷阱
10.1 动机：把“锁 + 等待”打包成语言级结构
10.2 条件变量的三个操作
10.3 Hoare vs Mesa 语义
10.4 有界缓冲区（监视器版）
10.5 读者—写者（Readers—Writers）：状态机式推理
10.6 语言层面的坑
10.7 关键数据结构与不变量
10.8 经典陷阱
11.1 动机：机制已备，策略待定
11.2 四个目标，彼此冲突
11.3 工作负载模型
11.4 FCFS / FIFO
11.5 Round Robin（RR）
11.6 优先级调度
11.7 关键数据结构与策略对比
11.8 经典陷阱
12.1 动机与机制：如果知道未来（SJF / SRTF）
12.2 用过去预测未来：指数平均
12.3 MLFQ（Multi-Level Feedback Queue）
12.4 工业案例：O(1) 调度器（Linux 2.6 早期）
12.5 CFS（Completely Fair Scheduler）
12.6 关键数据结构与调度器对比（含 PintOS 的落点）
12.7 经典陷阱
13.1 动机：三个容易混淆的概念
13.2 优先级反转（Priority Inversion）
13.3 Pathfinder（1997 火星探路者）
13.4 比例份额：Lottery 与 Stride
13.5 多核与亲和性
13.6 关键数据结构与不变量
13.7 经典陷阱
14.1 动机：实时 ≠ 快
14.2 周期任务模型
14.3 死锁的四个必要条件（必背）
14.4 资源分配图与处理策略
14.5 Dining Philosophers / 哲学家与筷子
14.6 关键数据结构与不变量
14.7 经典陷阱
15.1 动机：为什么不能“直接给物理地址”
15.2 从连续到非连续：三种方案的对比
15.3 分页的地址拆分
15.4 段页式（x86 的真实做法）
15.5 共享、写时复制、ASLR
15.6 关键数据结构与不变量
15.7 经典陷阱
16.1 动机与机制：把“大数组”变成“树”（两级页表）
16.2 TLB：翻译的缓存

16.3 缓存组织：为什么 TLB 与 cache 的设计互相牵制
16.4 缓存缺失四分类（讲义 15/16 反复用）
16.5 关键数据结构与不变量
16.6 经典陷阱
17.1 动机与机制：缺页的完整路径
17.2 精确异常（precise exception）
17.3 请求调页（Demand Paging）
17.4 `fork` 与 COW、按需加载、mmap
17.5 缺页处理中的“自由页框从哪来”
17.6 关键数据结构、不变量与经典陷阱
18.1 动机：为什么置换策略重要
18.2 四种基本算法
18.3 近似 LRU 的三种工程方案
18.4 没有硬件 use/dirty 位怎么办（软件模拟）
18.5 全局 vs 局部、分配策略、抖动
18.6 关键数据结构与不变量
18.7 经典陷阱
19.1 动机：12 个数量级的差异
19.2 硬件路径
19.3 数据传输：PIO vs DMA vs 中断/轮询
19.4 驱动结构：上半部 / 下半部
19.5 三种接口风格
19.6 设备分类
19.7 关键数据结构与不变量
19.8 经典陷阱
20.1 动机与机制：磁盘几何与延迟分解
20.2 磁盘调度算法
20.3 SSD：没有寻道，但有擦除
20.4 性能度量与排队论
20.5 关键数据结构与不变量
20.6 经典陷阱
21.1 全景
21.2 选择表（考场上最快的一张）
21.3 死锁检查清单
21.4 并发 bug 的四个典型形态
24.1 文件系统的四层结构（OSTEP/A&D 通用模型）
24.2 inode 与目录
24.3 空闲空间管理
24.4 缓冲缓存（buffer cache）
24.5 崩溃一致性：FSCK vs 日志
25.1 为什么分布式难
25.2 RPC：把调用变成消息
25.3 事务与 ACID
25.4 端到端论证（End-to-End Arguments）
25.5 NFS / AFS / Chord
26.1 Project 0：Pregame（个人，计入作业）
26.2 Project 1：User Programs
26.3 Project 2：Threads
26.4 Project 3：File Systems
28.1 一页纸清单（考前 30 分钟）
28.2 自测题（答案紧随）
29.1 课程与政策
29.2 每讲讲义（Lecture 1–19）
29.3 项目与文档
29.4 教材与补充
29.5 未获取/受限材料清单

第 1 章 课程概览：CS162 到底在教什么

1.1 一句话定位

CS162 是一门**“把操作系统拆开、再亲手装回去”**的系统课：前半学期用讲义把 OS 的四大抽象（线程、地址空间、文件、进程/通信）讲透，后半学期用 **PintOS** 这个只有几万行的教学内核，逼你在真实代码里实现用户程序、线程调度器和文件系统。

课程主页（Fall 2026）与政策页对课程目标的表述可以归纳为三句：

- 教**操作系统的设计**，以及“出现在其它计算机系统中的操作系统概念”（政策页原话：*the design of operating systems and operating systems concepts that appear in other computer systems*）；
- 覆盖多道程序系统（进程、IPC、同步）、内存分配（分段、分页）、资源分配与调度、文件系统、基础网络（socket、分层、API、可靠性）、事务、安全与隐私；
- 通过 PintOS 项目获得“**操作系统与分布式系统设计与实现**的深度体验”（政策页把项目称为 *essential to the course*）。

1.2 为什么这门课难，且难在哪

难点	具体表现	讲义/项目中的对应
并发	程序必须对**任意交错**都正确，测试无法穷举	讲义 6–9；Project 2
机制与策略分离	同一套上下文切换机制上要挂不同调度策略	讲义 10–13
硬件细节不可回避	页表、TLB、中断向量、段寄存器、iret	讲义 2、14–16；Project 1
大规模代码的调试	内核 panic 只有一串地址；需要 `backtrace` 与 GDB	PintOS 文档 Development 章
“能跑”不等于“对”	设计文档、口头答辩、代码质量都计分	政策页 Grading 段

讲义 1 里有一句话值得记住：*The question is not whether there are bugs but how serious are the bugs*（问题不是有没有 bug，而是 bug 有多严重）。这解释了为什么 OS 代码要“细致地处理错误、防御恶意或粗心的用户”。

1.3 先修要求（政策页原文归纳）

正式先修：**CS 61A、61B、61C、CS 70**（或等价课程）。课程默认你已经会：

- 数据结构：数组、链表、二叉树、哈希；
- 汇编语言编程；C 语言；用 GDB 调试 C；
- CPU cache 与存储层次；CS 61C 程度的虚拟内存；流水线与基本数字逻辑；
- CS 70 程度的随机变量与概率分布（队列论、 lottery 调度、EDF 可调度性都要用）。

政策页明确说：这些内容**不会在课上重讲**，TA 只会花少量时间复习。比形式先修更重要的是“调试大程序、设计抽象、计算性问题求解”的成熟度。

1.4 评分与考试（以政策页为准）

政策页给出的总分构成：

项目	占比	说明
考试（Exams）	36%	**三次期中，无期末**；每次等权（讲义 1 的暂定拆解为 12% × 3）
项目（Projects）	36%	三个项目**等权**；Project 0 因为个人完成，计入作业
作业（Homeworks）	18%	所有作业等权
参与（Participation）	10%	Discussion、Design Review、课堂参与

补充事实（均来自政策页/讲义 1）：

- 三次期中**都是线下闭卷**，只允许带限定张数的手写 cheat sheet（讲义 8 说“one page, both sides”；讲义 16 说第二次是“two pages, both sides, HANDWRITTEN”）。张数以当次考前通知为准。
- 项目与作业都通过 **GitHub + autograder** 提交；项目还有 **design document → design review（口头）→ code → final report** 的流程，并且有 **oral exam（口试）** 抽查你是否真的理解自己的代码。
- 项目必须**4 人一组**，政策页写明“不允许少于 3 人”，且要做 **peer evaluation**（互评），互评会实质影响分数。
- 讲义 1 明确写了 **“USE OF LLMs IS STRICTLY FORBIDDEN”**（严格禁止使用 LLM 完成作业/项目），并把“向 Claude 提问”类比为“向朋友要答案”。这是课程政策，不是建议。

1.5 日程骨架（来自主页 Schedule 表）

讲次	日期	主题	阅读
1–2	08/27, 09/01	什么是 OS；操作系统的四个基本概念	A&D Ch 1–2
3–4	09/03, 09/08	线程与进程；进程管理、文件与 I/O	A&D Ch 3–4
5–6	09/10, 09/15	文件 I/O 续、Socket 与 IPC；同步 1	A&D 5.1–5.3
7–8	09/17, 09/22	并发与互斥；锁实现、原子指令、futex	A&D 5.1–5.6
9–10	09/24, 09/29	信号量、监视器、读写者；调度 1	A&D Ch 5–7
11–12	10/01, 10/06	调度 2（案例）；调度 3（饥饿、优先级反转、实时）	A&D 7.1–7.6
13–14	10/08, 10/13	实时续、死锁；内存 1（VM、段、页表）	A&D Ch 8
15–16	10/15, 10/20	多级页表、TLB；TLB 续、请求调页	A&D Ch 9–10, 12
17–18	10/22, 10/27	请求调页策略；通用 I/O	A&D Ch 12
19–20	10/29, 11/03	驱动与存储性能；文件系统 1（性能、排队论、设计）	A&D Ch 11–13
21–22	11/05, 11/10	特别讲座（主题以主页为准）；文件系统 2（设计与案例）	A&D Ch 13–14
23–24	11/12, 11/17	文件系统 3（buffer cache、可靠性、事务）；分布式 1	—
25–26	11/19, 11/24	分布式 2（决策、TCP/IP）；分布式 3（RPC、NFS）	—

讲次	日期	主题	阅读
27–28	12/01, 12/03	分布式 4（NFS/AFS/Chord）；特别讲座（主题以主页为准）	—

考试：Midterm 1 = 10/02（周五）18:00–21:00，Gateway 1210/1220；Midterm 2 = 10/30 同时间同地点；Midterm 3 = 12/04 同时间同地点。

1.6 四个项目 + 六个作业总览

编号	名称	形式	核心内容	关键日期（主页）
Project 0	Pregame	**个人**（计入作业）	用 GDB 走一遍 PintOS 启动 → 加载用户程序 → 崩溃 → 修一行 `process.c` → 实现 `practice` 系统调用	09/01 发布，09/11 截止
Project 1	User Programs	小组	参数传递、9 个文件系统调用 + `practice/halt/exit/exec/wait/fork`，用户内存安全检查	09/12 发布，设计文档 09/20
Project 2	Threads	小组	高效 `timer_sleep`、严格优先级调度 + 优先级捐赠、用户级线程与用户级锁/信号量	10/09 发布，设计文档 10/17
Project 3	File Systems	小组	64 块 buffer cache、可扩展文件（索引 inode）、子目录与路径解析、细粒度同步	11/05 发布，checkpoint 11/13，设计文档 11/14
HW 0	Introduction	个人	环境、git、autograder、GDB、`objdump`、Unix 工具	09/07 截止
HW 1	List	个人	在 Linux 用户态用 Pintos 的 `list` 数据结构实现线程安全链表（`lwords` / `pwords`）	09/20 截止
HW 2	Shell	个人	实现 shell：目录命令、程序执行、路径解析、重定向、管道、信号、进程组、前后台	10/05 截止
HW 3	HTTP	个人	实现 HTTP/1.0 服务器：GET、响应头、错误码、目录列表、代理	10/14 截止
HW 4	Memory	个人	实现 `sbrk` 与用户态 `malloc/realloc/free`	10/29 截止
HW 5	Map Reduce	个人	C 版或 Rust 版 MapReduce	11/20 截止
HW 6	Raft	个人	C 版或 Rust 版 Raft	12/11 截止
RPC Lab	RPC	个人	协议规格、RPC 实现、发 RPC、KV store（C 版或 Rust 版）	11/07 截止

> 注：主页把 HW3 标为“HTTP (C version) / (Rust version)”，HW5/HW6 与 RPC lab 同样提供 C/Rust 双版本。作业编号与主题来自主页 Schedule 表；具体任务书在 `https://cs162.org/static/hw/<name>/`。

1.7 与 CS111 / CSAPP / CS144 的定位差异

维度	CS162（Berkeley）	CS111（Stanford）	CSAPP（CMU 15-213）	CS144（Stanford）
主线索	四大抽象 + **真实内核实现**	原理 + 用户态模拟（线程库、FUSE）	程序视角：编译、链接、异常控制流、存储层次	协议栈实现
代码规模	修改一个完整教学内核 PintOS	在用户态“造一个小 OS”	若干 lab（malloc、shell、proxy、cache）	从 ByteStream 到 TCP
并发深度	内核锁、关中断、futex、优先级捐赠	监视器、用户态调度器	线程级并行、同步基础	事件驱动、单线程并发
内存深度	页表/段/TLB/请求调页/置换策略全套	类似但更偏原理	cache 与 malloc 极深，页表较浅	几乎不涉及
文件系统	索引 inode、buffer cache、目录、崩溃一致性	V6 文件系统 + 日志	无	无
分布式	事务、RPC、NFS/AFS、Chord（讲义 24–27）	无（有 Flash/VM）	无	网络为主

一句话：**CSAPP 教你“程序如何被机器执行”，CS111 教你“OS 原理长什么样”，CS162 要求你“把 OS 写出来并通过 autograder”**。三者的内存/并发章节高度互补：CSAPP 给你 cache 与链接的直觉，CS111 给你策略比较，CS162 给你“这段代码在真实内核里会不会 panic”。

第 2 章 讲义 1：什么是操作系统（What is an Operating System?）

2.1 动机：为什么需要一层“中间人”

讲义 1 开场先问“OS 到底做什么”，学生最容易给出的答案是：内存管理、I/O 管理、CPU 调度、通信、多道程序、文件系统、多媒体支持、用户界面、浏览器…… 讲师的回答是：**没有 universally accepted definition（没有普遍接受的定义）**。两个可用的近似是：

- “厂商发货时带的一切” —— 近似但差异巨大；

- “**永远在运行的那一个程序**就是内核（kernel）”，其余是系统程序或应用程序。

真正有用的是下面这句功能性定义：

> 操作系统是一层**特殊软件**，它为应用软件提供访问硬件资源的通道：把复杂的硬件设备抽象成方便的接口，对共享资源提供**受保护的**访问，并提供安全、认证与通信。

2.2 三种角色：Referee / Illusionist / Glue

讲义用三个词贯穿全课，这三个词也是期末复习的骨架：

```
app1  app2  app3          ← 应用
   | 系统调用 / 异常 / 中断（受控入口）
Operating System: Referee（裁判）| Illusionist（幻觉师）| Glue（胶水）
   | 物理机器接口（ISA + 设备寄存器）
Processor | Memory | Storage | Network
```

角色	含义	典型机制	讲义中的例子
Referee（裁判）	管理资源的保护、隔离与共享；资源分配与通信	双模式、地址翻译、抢占	一个进程不能读写另一个进程的内存
Illusionist（幻觉师）	提供干净易用的抽象，掩盖硬件限制	线程、地址空间、文件、socket	“无限内存、独占机器”
Glue（胶水）	提供公共服务的统一接口	存储、窗口系统、网络、授权	“一切皆文件”

2.3 抽象映射表（背下来）

硬件	OS 抽象	讲义表述
Processor（核）	**线程（Thread）**	执行上下文
Memory（物理内存）	**地址空间（Address Space）**	一组可访问地址 + 关联状态
Disks / SSD	**文件（File）**	命名数据 + 元数据
Network	**Socket**	双向字节流端点
Machine	**进程（Process）**	受保护的地址空间 + 一个或多个线程

课件里还有一句方法论：对 OS 的**任何一个子系统**，都要问两个问题——

1. 要处理什么**硬件接口**（physical reality）？
2. 要提供什么**软件接口**（nicer abstraction）？

这两问是后面所有章节（调度、VM、FS、网络）的统一分析框架。

2.4 双模式操作：保护的全部基础

硬件至少提供两种模式：**Kernel mode（内核/监督模式）** 与 **User mode**。用户态下被禁止的操作包括：

- 修改页表基址寄存器（x86 的 CR3）；
- 关中断；
- 直接与硬件设备交互（in/out 指令、DMA 寄存器）；
- 写内核内存。

用户态进入内核态只有三条受控路径：**系统调用（syscall）**、**中断（interrupt）**、**异常（exception/trap）**。x86 实际有 4 个 ring（2 bit），多数 OS 只用 Ring 0 与 Ring 3；新处理器还有 hypervisor 用的“Ring −1”。

```
用户态                                          内核态
  │  int $0x30 / syscall  ──────────────►  系统调用处理程序
  │  定时器中断 / 设备中断 ──────────────►  中断处理程序
  │  缺页 / 除零 / 保护违例 ──────────────►  异常处理程序
  │  ◄──────────  iret（恢复 CS:EIP, EFLAGS, SS:ESP, PL）
```

不变量：任何时刻 CPU 处于且仅处于一种模式；模式的改变只能通过上述受控入口或 `iret` 完成。这条不变量一旦被破坏（例如内核错误地执行了用户提供的栈指针），整个保护模型就失效——这正是 Project 1 “检查用户指针”任务的由来。

2.5 复杂度论证：为什么 OS 必须“工程化”

讲义用一组数字建立“复杂度”的直觉：

- Linux 2.2.0 到 Linux 3.1、Windows Vista/7、Office 2013、现代汽车、Facebook 的代码行数量级逐级上升（课件给的是一张百万行级柱状图）；
- 现代 CPU 有百亿级晶体管：讲义举的 2023 年 Intel Sapphire Rapids 为“最多 60 核 / 120 线程每 socket、最多 4 个 chiplet、8 socket 直连可形成 480 核共享内存域、每核 80KB L1 / 2MB L2、最多 112.5MB L3、最多 4TiB DDR5”；
- **第三方设备驱动是最不可靠的代码之一**（"poorly written by non-stake-holders"），可能把数据从受保护的内核空间抽走（讲义举 Heartbleed 作为“越界读”的著名案例）。

结论：不可能对所有环境组合做测试，所以 OS 的写法必须是“**防御式**”的：每个入口都验证参数、每个资源都有归属、每条错误路径都释放资源。

2.6 历史与技术趋势（考点的“背景板”）

三个阶段：**1940s–60s** 硬件贵、人便宜 ⇒ 批处理、提高机器利用率（1962 IBM 7094 的内存重定位与保护催生多道程序与内核）；**60s–80s** 硬件变便宜 ⇒ 分时、文件系统、响应时间；**90s 至今** 网络无处不在 ⇒ 连接、规模、可预测性。

讲义还给出两个“定律”的收尾：Moore's Law 描述的晶体管密度增长在 2016 年前后被宣布“死亡”，但厂商用 3D 堆叠、chiplet、专用加速器继续提升；单核性能年增长率从 52%/年 降到 25%/年 再降到个位数，**于是并行成为唯一出路**（“The World is Parallel”）。这正是“为什么这门课要讲并发与多核调度”的物理原因。

2.7 关键数据结构与不变量（本章层面）

概念	载体	不变量
模式位	CPU 状态寄存器（如 x86 的 CS.RPL / EFLAGS）	用户态不能改 CR3、不能关中断
内核栈	每线程一段内核栈	进入内核后**绝不复用用户栈指针**
系统调用表	内核中的函数指针数组	编号越界必须被拒绝
中断向量表（IDT）	硬件寄存器指向的表	每个向量都有合法处理程序，否则 double/triple fault

2.8 经典陷阱

1. **把“OS = 内核”当成定义**：讲义明确说这是近似。用户态的系统库（libc）在很大程度上“是 OS 的一部分”，但它运行在用户态。
2. **以为用户态可以关中断**：不能。所以“关中断实现锁”只能用于内核自身（讲义 7–8）。
3. **忘记 `iret` 会恢复优先级**：写上下文切换代码时若手工恢复寄存器而漏掉 `iret`，会以用户态继续执行内核代码（或反之），表现为莫名的保护违例。
4. **把“复杂度”当借口不做参数检查**：Project 1 的 robustness 测试专门打这类漏洞（空指针、指向内核的指针、跨页边界的 4 字节参数）。
5. **相信课程允许用 LLM**：讲义 1 白纸黑字禁止；且课程会与往年提交、网上答案比对，并口试要求你解释代码。

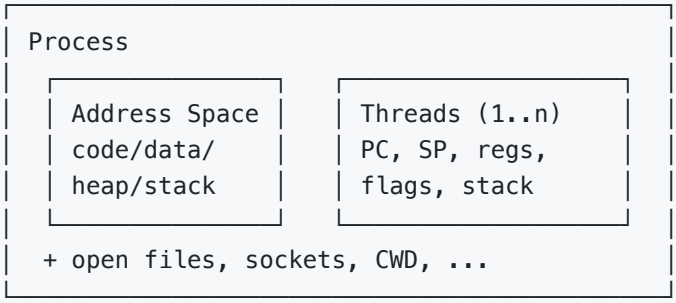
第 3 章 讲义 2：操作系统的四个基本概念（Four Fundamental OS Concepts）

3.1 动机：把“魔法”拆成四个可实现的部件

讲义 2 的标题就是全课的骨架。四个概念是：

1. **Thread（线程）**：执行上下文；
2. **Address Space（地址空间）**：可访问地址的集合 + 关联状态；
3. **Process（进程）**：受保护的地址空间 + 一个或多个线程；
4. **Dual-mode operation / Protection（双模式与保护）**：只有“系统”能访问某些资源。

课程反复强调：**Thread 封装并发（active），Address Space 封装保护（passive）**。



3.2 线程：一个“虚拟核”

- **执行中的线程**：其状态驻留在处理器寄存器中（含中间值、指针）。
- **被挂起的线程**：状态不在寄存器中，而是保存在内存里的 **TCB（Thread Control Block）**；至少包含 PC、SP、通用寄存器、执行标志。
- 一个线程“在运行”意味着：PC 指向它的下一条指令，SP 指向它的栈。

切换的本质（讲义原话的复述）：

```
T1: vCPU1 在真核上，vCPU2 在内存里
    OS 运行（怎么运行的？中断/系统调用）
    把 PC/SP/regs 存入 vCPU1 的 TCB
    从 vCPU2 的 TCB 载入 PC/SP/regs，跳到 PC
T2: vCPU2 在真核上，vCPU1 在内存里
```

触发切换的事件：**定时器、自愿 yield、I/O 完成、其它中断**。

3.3 地址空间与两种“翻译”方案

地址空间 = 程序可访问的地址集合 + 这些地址对应的状态。读/写一个地址可能有四种效果：像普通内存、忽略写、触发 I/O（memory-mapped I/O）、抛异常（fault）。

从“无翻译”到“有翻译”的演化，讲义给了三级台阶：

3.3.1 无翻译的多道程序（Loader/Linker 重定位）

```
物理内存: [ OS/向量表 | program A | program B ]
          链接/加载时把所有地址改写成“加载地址 + 偏移”
```

问题：程序能碰 OS、能碰彼此；没有保护；加载位置一变就要重新 relocate。

3.3.2 Base & Bound（基址+界限）

```
虚拟地址 VA ──► [ 比较 VA < Bound ? ] ──► PA = VA + Base ──► 物理内存
                          │否
                          └──► 异常（段错误）
```

优点：地址通路上只加一次加法（无逐页查表）；缺点：**一个进程只能是一块连续区域**，无法分别放置 code/data/stack/heap，难以共享，且产生**外部碎片**。

3.3.3 分段（Segmentation）

```
虚拟地址 = [ Segment ID | Offset ]
                  │           │
                  ▼           ▼
        段表项: Base, Limit, V/N
        PA = Base + Offset    （并检查 Offset < Limit）
```

- 逻辑视图：多个独立段（code、data、stack，x86 还可用 `mov [es:bx], ax` 在指令里指定段）；
- 段表在 CPU 内（很小），切换进程时换段寄存器；
- 仍要“把可变大小的块塞进物理内存” → 外部碎片 + 内部碎片；换出/换入代价大（整段）。

3.3.4 分页（Paging）

```
虚拟地址:  [ VPN (20b) | Offset (12b) ]        物理地址: [ PPN | Offset ]
                  │
                  ▼
        页表（每进程一张，位于物理内存）:
        ┌─────────┬─────┬─────┬─────┐
        │ VPN     │ V   │ R   │ W   │ → PPN
        └─────────┴─────┴─────┴─────┘
```

- 物理内存被切成固定大小的 **frame**，任意虚拟页可放入任意 frame；
- **offset 原样复制**，只有页号被翻译；
- 每个 PTE 还带权限位（valid / read / write / user）；
- 碎片从“外部”变成“内部”（一个页内未用满）。

不变量：页表本身只能被内核修改；所有页表页必须只映射在内核态可访问的区域（否则用户可改写自己的翻译，保护全失）。

3.4 进程：为什么需要它

需求	机制
可靠性：一个 bug 只破坏自己的内存	地址翻译
安全与隐私：进程不能读写别人的数据	翻译 + 双模式
公平：强制 CPU/磁盘份额	调度 + 配额
通信	显式 IPC（pipe/socket/共享内存）

讲义点出**保护与效率的根本权衡**：进程内通信容易（共享地址空间），进程间通信困难（必须显式 IPC）。这就是“为什么要有多线程”的答案之一：既想要并行，又想要共享。

3.5 三种“进入内核”的方式（必背）

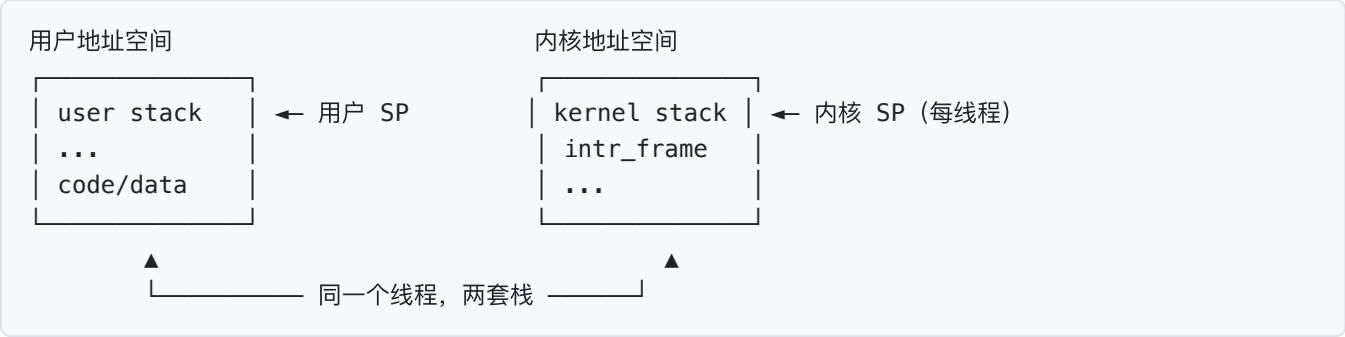
类型	触发	同步/异步	例子	处理程序位置
System call	程序主动 `int` / `syscall`	同步	`read()`、`fork()`	系统调用表（按编号）
Interrupt	外部设备/定时器	异步	网卡收包、timer tick	中断向量表（按 IRQ）
Trap / Exception	指令执行导致	同步	缺页、除零、保护违例	异常向量表（按异常号）

三者都是 unprogrammed control transfer（非程序控制的控制转移）。讲义特别强调它们“看起来像函数调用，但你没有目标地址”——这正是 RPC 思想的雏形。

3.6 内核栈与用户栈：双栈模型

进入内核时，硬件/软件要切换到**内核栈**，原因：

- 不能信任用户 SP（可能是空指针、可能指向内核）；
- 不能信任用户栈上的数据（其它线程可能篡改）；
- 内核需要连续、可信的工作区。



PintOS 的实现把 **TCB 与内核栈放在同一页**：`struct thread` 在页首，栈从页尾向下生长。这条设计约束（第 26 章会展开）导致两个硬性规则：`struct thread` 不能太大（建议远小于 1KB），内核函数不能在大栈上放巨大数组。

3.7 关键数据结构与不变量

结构	字段（PintOS 对应）	不变量
TCB / `struct thread`	`tid, status, stack, priority, elem, allelem, pcb, magic`	每线程唯一 tid；`magic` 用于检测栈溢出
PCB / `struct process`	pid、子进程表、fd 表、退出状态	每个用户进程一个；父进程负责回收
页表	PTE 数组/多级	用户态不可写；进程切换必须换 CR3
中断向量表	256 项（x86）	每项必须有效

3.8 经典陷阱

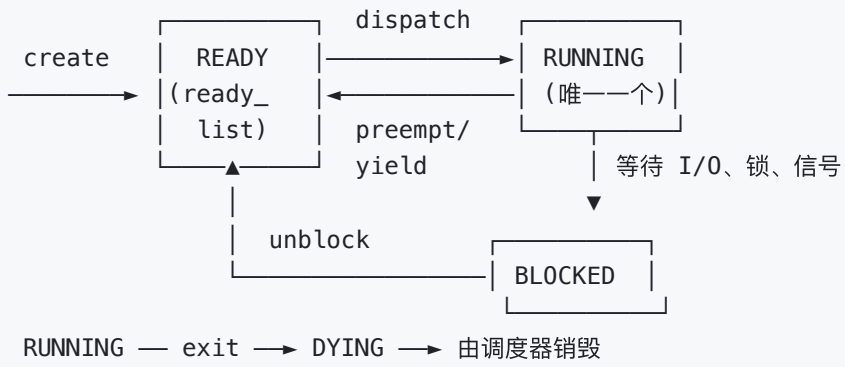
1. **以为 base&bound 能支持“代码/数据/栈分离”**：不能，它只有一个连续区间；这正是引入分段/分页的动机。
2. **把“进程 = 程序”**：进程是“运行中的程序实例”，同一二进制可以对应多个进程（讲义 14 的 “different processes running same binary” 还会用到这点：代码段可共享）。
3. **在用户态直接改页表**：硬件禁止；若内核把页表页映射为用户可写，保护即失效。
4. **混淆 “concurrent” 与 “parallel”**：单核上两个线程可以并发（交错）但不可能并行；讲义 3 用 `ComputePI` + `PrintClassList` 的例子说明并发能让 I/O 与计算重叠。
5. **以为中断处理程序可以睡眠**：讲义 4 指出中断处理程序“invoked with interrupts disabled，run to completion，no waits”；重活应交给内核线程（上半部/下半部）。

第 4 章 讲义 3：线程与进程抽象（Threads and Processes）

4.1 动机：把“一个核”复用成“很多个核”

单核机器上如何让多个任务“同时”推进？答案是**时间复用（multiplex in time）**：线程是“虚拟核”，OS 在这些虚拟核之间快速切换，使每个线程都像独占一个核。讲义 3 的目标很具体：讲清**线程是什么、不是什么、如何写多线程程序、有哪些替代方案**。

4.2 线程的三状态模型与生命周期



PintOS 的 `enum thread_status` 正是这四态：`THREAD_RUNNING / READY / BLOCKED / DYING`。**不变量**：任一时刻**最多一个**线程处于 RUNNING（单核）；`thread_current()` 返回它。

进程的生命周期讲义给的是五态（new / ready / running / waiting / terminated），比线程多一个 `new`，本质相同：**未运行 ⇒ PCB 挂在某个队列上**，不同事件用不同队列（每个设备/条件一个队列），每个队列可以用不同调度策略。

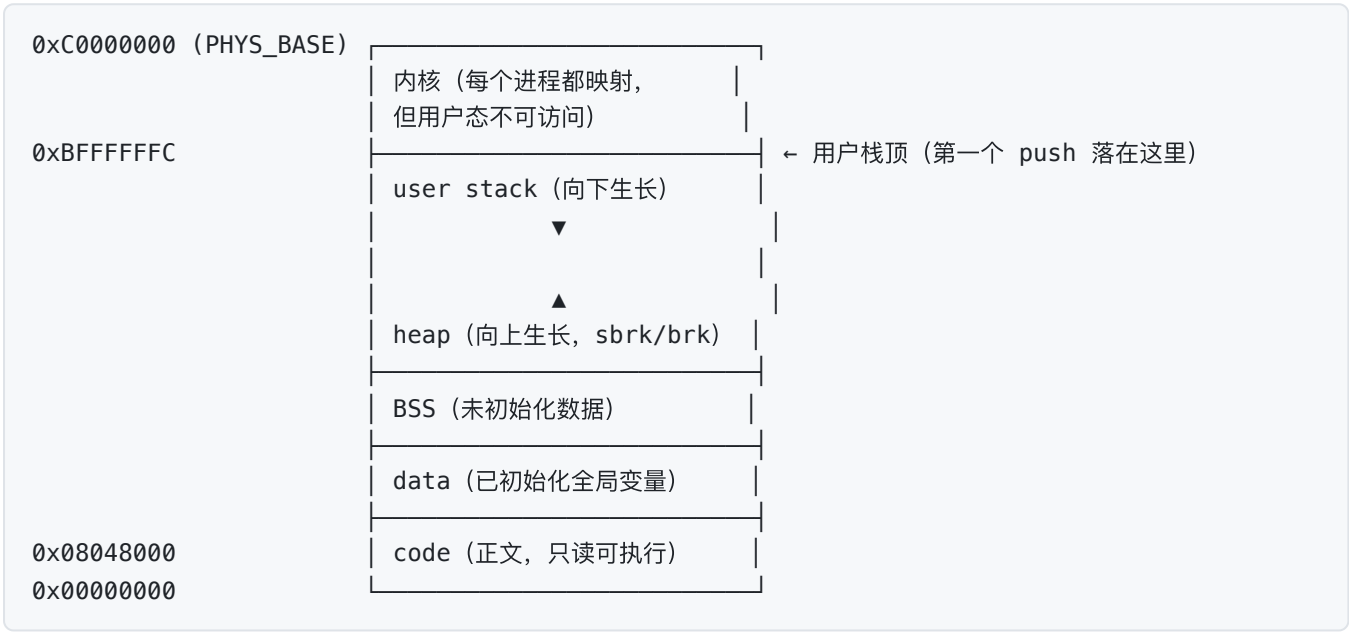
4.3 进程创建：fork / exec / wait 的语义

调用	语义	返回值/效果
`fork()`	复制当前进程（地址空间、fd 表）	父进程得到子 pid；子进程得到 0；失败 <0
`exec()`	用新程序替换当前进程的 code/data	成功不返回；失败返回 −1
`wait()`	等待子进程结束并取回状态	返回子 pid 与 status
`exit()`	终止进程，释放资源	不返回

讲义 3 的 fork 例子特意留了一个“陷阱问题”：如果在 fork 前后加 `sleep()` 会怎样？答案是**父子进程的输出顺序不确定**——这就是并发的第一课：**没有同步就没有顺序保证**。

第一个进程从哪来：内核启动时创建 **INIT** 进程，之后所有进程都由别的进程创建（“chicken and egg”的答案）。PintOS 里对应 `threads/init.c` 的 `main()` 调 `run_task()` → `process_wait(process_execute(task))`。

4.4 地址空间布局（x86 32 位，PintOS）



这张图在 Project 1（参数传递）和 Project 2（每线程独立用户栈）里都会直接用到。

4.5 上下文切换：到底保存了什么



关键认知：**只需要保存“被调用者保存寄存器”和 SP**，其余寄存器在中断发生时已经由硬件/入口代码压在栈上。切换的代价（讲义 7 给的 Linux 数字）：

切换类型	典型耗时	额外工作
同进程内线程切换	~100 ns	存/取寄存器，换栈

切换类型	典型耗时	额外工作
跨进程切换	3–4 μs	还要换地址空间（CR3）、刷 TLB
切换频率	10–100 ms 一次	太频繁 ⇒ 开销占比升高

4.6 用户级线程 vs 内核级线程

维度	用户级线程（green threads）	内核级线程
谁调度	用户态库（无内核参与）	内核调度器
切换成本	极低（几十 ns，无陷阱）	较高（陷入内核）
阻塞系统调用	**一个线程阻塞 ⇒ 整个进程阻塞**	只阻塞该线程
多核并行	不能（一个内核线程承载多个用户线程）	能
PintOS Project 2 的选择	**不允许**（明确要求 1:1 映射）	采用

讲义 6 还提到 **SMT / Hyperthreading**：硬件复制寄存器状态，让一个物理核同时持有多个执行上下文，“像调度独立线程一样调度它们”，但加速是**次线性**的。

4.7 关键数据结构与不变量

- `struct thread`：`tid`（全局唯一、单调递增）、`status`、`stack`（不在 CPU 上时保存的 SP）、`priority`（0–63）、`elem`（挂 ready_list 或信号量等待队列，**二选一**）、`allelem`（挂 all_list）、`pcb`、`magic`。
- **不变量**：`elem` 同一时刻只能在一个链表里；`status == BLOCKED` 的线程一定在某等待队列里；`status == READY` 的线程一定在 `ready_list` 里。
- 线程栈溢出会覆盖 `struct thread` 的字段，`magic` 被破坏 ⇒ `thread_current()` 断言失败。这是 PintOS 里最常见的 panic 之一。

4.8 经典陷阱

1. **多线程共享局部变量**：讲义 3 的示例代码让多个线程写同一个变量，结果依赖调度顺序（“Is this code safe/correct? No”）。
2. **以为 `fork()` 后父子共享内存**：不共享（写时复制是优化，语义仍是各自独立）。PintOS 的 `fork` 要求复制地址空间与 fd 表，但**不要求** COW。
3. **忘记 `wait()` 会造成僵尸**：PintOS 要求“无论父进程是否 wait，子进程资源都必须释放”（第 26 章 P1 的难点）。

4. **在中断处理程序里做耗时工作**：会拉高中断延迟；正确做法是“上半部记账、下半部干活”。
5. **用 `sleep()` 解决竞态**：sleep 只能改变概率，不能提供保证；讲义后面会用“Too Much Milk”证明这一点。

第 5 章 讲义 4：进程管理、文件与 I/O（Process Management, Files and I/O）

5.1 动机：进程之间如何“说话”

默认情况下进程之间**没有任何数据通路**（这也是保护的意义）。因此通信必须**显式建立**。讲义 4 的路线是：先用“一切皆文件”统一接口，再把 pipe/socket 也套进这个接口。

5.2 “一切皆文件”（Everything is a File）

Unix/POSIX 的核心设计：**同一个接口**（`open/read/write/close`，外加 `ioctl`）覆盖

- 磁盘上的普通文件；
- 设备（终端、打印机）；
- 网络（socket）；
- 本地 IPC（pipe、Unix domain socket）。

讲义特别指出这在提出时是“radical idea”，出自 Ritchie 与 Thompson 的 UNIX 论文（资源页有链接）。代价是：所有东西都被迫是“字节流”，设备特有的操作只能塞进 `ioctl`。

5.3 文件的两层抽象：fd 与 FILE*

```
应用程序
  │  FILE *f = fopen(...)          ← 用户态缓冲（libc）
  │  fread/fwrite/fseek/fclose
  ▼
libc（用户态缓冲，按块读写）
  │  read(fd, buf, n) / write(...)  ← 每次都是系统调用
  ▼
内核：fd 表 → open file description（文件偏移、flags、引用计数）→ inode → 设备
```

维度	低级 I/O（fd）	高级 I/O（FILE*）
标识	`int fd`（进程内索引）	`FILE *`（用户态结构）

维度	低级 I/O（fd）	高级 I/O（FILE*）
缓冲	无（内核可能有 page cache）	有，用户态缓冲区
系统调用次数	每次 read/write 一次	摊薄到块大小
可见性	立即对其它进程可见	需要 `fflush` 才落到内核
定位	`lseek(fd, off, whence)`	`fseek` + 用户态缓冲（`ftell` 不等于内核偏移）

为什么给用户整数而不是内核指针：安全（不能伪造/解引用）、可移植（fd 表可以做权限检查）、可复用（fd 号可以重用）。

5.4 dup / pipe / 重定向：shell 的实现基础

```
int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fd, STDOUT_FILENO);   /* 让 fd 1 指向 out.txt 的 open file description */
close(fd);                 /* 原来的 fd 不再需要 */
execvp("ls", argv);        /* ls 的输出自动进文件 */
```

`dup2(old, new)` 的关键语义：**new 与 old 共享同一个 open file description**（因此共享文件偏移），只是 fd 号不同。`pipe(fd[2])` 创建一个单向队列：写 `fd[1]`，读 `fd[0]`。

```
cat hello.txt | grep World
  ┌─────────┐  stdout(fd1)   pipe    stdin(fd0)  ┌─────────┐
  │  cat    │ ─────────────▶ ═══ ─────────────▶ │ grep    │
  └─────────┘                                   └─────────┘
     fork+dup2 把 cat 的 fd1 接到 pipe 写端；把 grep 的 fd0 接到读端
```

不变量：pipe 是单向的；读端在所有写端关闭后读到 EOF；写端在读端关闭后写会得到 `SIGPIPE` / `EPIPE`。

5.5 阻塞语义与调度器的交互

- `read()` 在数据未就绪时**阻塞**：进程被放入该设备的等待队列，调度器运行别的线程；数据到达后中断处理程序唤醒它。
- `write()` 在缓冲区满时也可能阻塞（例如 pipe、socket）。
- 这解释了“为什么 I/O 密集程序需要并发”：阻塞点正是 CPU 可以交给他人的时刻。

5.6 关键数据结构与不变量

结构	内容	不变量
fd 表（每进程）	fd → `struct file *`	0/1/2 默认打开；`fork` 复制表项；`exec` 保留（除非 CLOEXEC）
open file description	偏移、访问模式、引用计数	多个 fd 可指向同一个描述（dup/fork）
inode	元数据 + 数据块位置	名字与 inode 分离（硬链接的本质）
pipe	环形缓冲区 + 读写端计数	读写互斥；空/满时阻塞

5.7 经典陷阱

1. **忘记检查返回值**：讲义原话“Systems programmers should always be paranoid!”；`read` 可能短读，`write` 可能部分写。
2. **混淆 `fflush` 与 `fsync`**：前者只把用户缓冲交给内核，后者才要求内核落盘。
3. **在信号处理函数里调用非 async-signal-safe 函数**（如 `printf`）：会死锁或崩溃；HW2 的 shell 要用 `sigaction` + `waitpid` 正确处理 `SIGINT`。
4. **忘记关闭 fd**：PintOS 的 `exec-multiple` / `multi-oom` 类测试会因此耗尽资源。

第 6 章 讲义 5–6：Socket 与 IPC（Sockets and IPC）

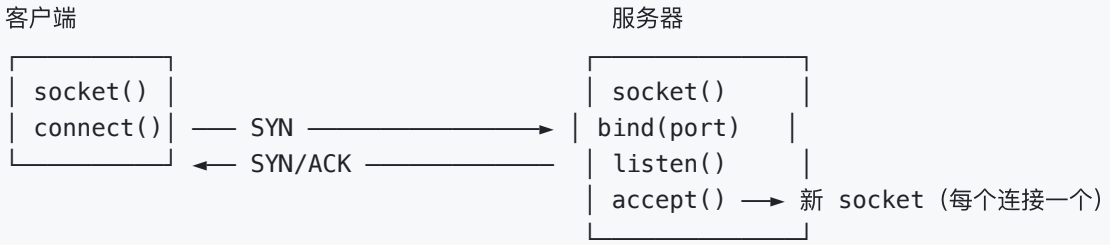
6.1 动机：跨机器的“文件 I/O”

文件系统提供“永久对象 + 结构化命名空间”，但它是**持久**的、**有名字**的、**可重复读**的。通信需要的是另一种东西：**瞬时的、连接的、一次性的队列**。讲义 5 的问题串很精彩：

- 文件是“写一次、读一次”吗？不是，文件可重复读，读到的取决于读的时刻；
- 通信更像**队列**：数据写一次、被消费一次；
- 那么“open 的类比是什么？命名空间是什么？两端如何在时间上对上？”

答案就是 **socket 接口**：把“跨世界的通信”做成“文件 I/O”。

6.2 Socket 与连接的表示



概念	说明
Socket	网络连接的**一个端点**的抽象；有 fd；`write` 追加到发送队列，`read` 从接收队列取
连接	两个 socket 通过网络的配对；“两个队列”（每方向一个）
监听 socket	只用于 `accept`，不能读写
5-tuple	(源 IP, 源端口, 目的 IP, 目的端口, 协议) 唯一标识一条 TCP 连接

端口范围（讲义给的分段）：0–1023 是 well-known（需 root 绑定）；1024–49151 是 registered（IANA 分配）；49152–65535 是 dynamic/ephemeral（客户端由 OS 自动分配）。

6.3 服务器并发模型

模型	结构	优点	缺点
迭代服务器	一个连接处理完再 accept 下一个	最简单	并发度 1，长连接会饿死其他人
每连接一个进程	`fork()` 后子进程处理	隔离性好	进程创建贵；共享状态难
每连接一个线程	`pthread_create`	比进程便宜	线程数爆炸；需同步
线程池	固定 N 个 worker + 队列	有界资源、可预测	需要队列与背压
事件驱动（select/epoll）	单线程 + 非阻塞 I/O	高并发、无切换	编程模型复杂（状态机）

讲义 5/6 的顺序是“迭代 → 每连接进程 → 每连接线程 → 线程池”，并强调一个常见 bug：多个子进程/线程共享同一个输出 fd 时，**写操作可能交错**（要用 `write` 一次性写出整块，或加锁）。

6.4 IPC 全景

机制	方向	范围	是否有名	典型用途
pipe	单向	同机、有亲缘关系	无（靠继承）	shell 管道

机制	方向	范围	是否有名	典型用途
FIFO（named pipe）	单向	同机任意进程	有（文件系统路径）	简单守护进程通信
Unix domain socket	双向	同机	有（路径）	本机高性能 IPC
TCP socket	双向	跨机	有（IP:port）	网络服务
共享内存	双向	同机	有（shm 名）	大数据量、需自行同步
消息队列	双向	同机/跨机	有	解耦、缓冲

6.5 关键数据结构与不变量

- 每条连接在内核里有发送队列与接收队列；`write` 在队列满时阻塞（或返回 EAGAIN）。
- `accept()` 返回**新的 fd**；监听 fd 继续存在。
- **不变量**：一个 TCP 连接由 5-tuple 唯一确定；同一端口可以同时有监听 socket 与多条已建立连接（靠四元组区分）。

6.6 经典陷阱

1. **忘记处理短写**：`write(fd, buf, n)` 可能只写一部分，必须循环。
2. **`accept` 后不关闭不需要的 fd**：fd 泄漏，最终 `EMFILE`。
3. **多线程共享 socket 却不加锁**：两个线程同时 `write` 会交错（除非用 `sendmsg` 或加锁）。
4. **忽略字节序**：网络协议用大端；`htons/htonl` 不能省。
5. **把 `read` 返回 0 当成错误**：0 表示对端关闭（EOF），这是正常路径。

第 7 章 讲义 6–7：并发、互斥与临界区（Concurrency and Mutual Exclusion）

7.1 动机：并发是“送命题”，也是“送分题”

讲义 6 的后半段切换到并发，理由是：一旦有了多个线程共享地址空间，**任何共享可变状态都可能被破坏**，而且“调度器可以产生任意交错，如果它能产生最坏交错，它就一定会”。这句话是全课并发的“悲观假设”，后面所有同步机制都是为了在这个假设下仍然正确。

并发为什么值得？讲义给的服务器例子：单 CPU 上仍然希望**重叠 I/O 与计算**；用线程可以“不把代码拆成事件驱动的碎片”。否则你就要把一个请求处理函数改写成几百个回调片段。

7.2 原子性：并发推理的地基

- **原子操作**：不可被中断、中间状态不可被他人观察的操作。
- 硬件保证的原子性很弱：**单字的 load/store 是原子的**（大多数机器）；`x++` **不是**（load–modify–store 三步）。
- 讲义用一个经典例子：初值 `y = 12`，两个线程分别执行 `x = y+1` 与 `y = y+1`，`x` 的可能值包括 12、13、14 等，取决于交错。
- 更狠的例子：两个线程分别写 32 位字的两半（在允许非对齐写的机器上），可能写出“两个线程都没写过的值”。

结论：仅靠 load/store 原子性，构造不出可靠的同步；必须要么有硬件原子指令，要么有关中断。

7.3 临界区与互斥：术语精确化

术语	定义
互斥（Mutual Exclusion）	保证同一时刻只有一个线程执行某段代码
临界区（Critical Section）	那段“只能一个人进”的代码
锁（Lock）	实现互斥的对象；`acquire` 等待并占有，`release` 释放
忙等（Busy Waiting / Spin）	等待时仍消耗 CPU
饥饿（Starvation）	某线程长期无法推进（但理论上可能推进）

“锁”的接口约定（讲义反复强调）：

- `acquire(&lock)`：等到锁空闲，然后标记为占用；返回后调用者**持有**锁；
- `release(&lock)`：标记为空闲；**只能由持有者调用**；
- 两个操作必须原子；`acquire` 与 `release` 必须配对。

7.4 Too Much Milk：为什么“只靠读写”不行

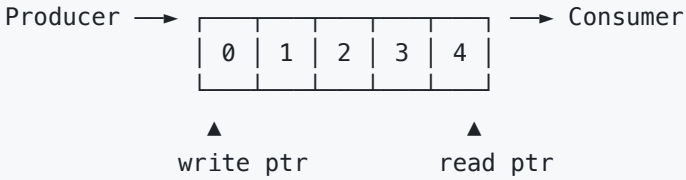
问题：室友之间不能同时买牛奶；只用“留纸条/看纸条”（即原子的 load/store）来协调。

尝试	代码思想	失败原因
1	看没有牛奶 → 买	两人同时看到没牛奶
2	先留纸条，再看牛奶	一人留条未买，另一人看到条就不买 ⇒ **可能没人买**
3	两张纸条（A、B 各一张），按固定顺序检查	正确但复杂；A、B 代码不同；等待时**忙等**；线程数一多不可扩展

尝试	代码思想	失败原因
Lamport Bakery	取号、按号排队	正确、可扩展，但仍忙等，且实现复杂

讲义的评价非常直接：解法 3 “works, but it's really unsatisfactory”——太复杂、难证明、不可扩展、忙等。**这就是为什么需要硬件提供更高级的原语。**

7.5 生产者—消费者与有界缓冲区



三类约束必须分开处理（这是本章最重要的方法论）：

1. **互斥**：队列本身是共享数据，入队/出队必须原子；
2. **调度约束 1**：缓冲区满时生产者必须等；
3. **调度约束 2**：缓冲区空时消费者必须等。

用**锁 + 轮询**可以写，但会忙等且难以表达“等哪个条件”；用**信号量**可以写（第 9 章），用**监视器**最清晰（第 10 章）。讲义用“可乐机”作类比：机器容量有限（`emptySlots`），机器里可乐数有限（`fullSlots`），投币/取货要互斥（`mutex`）。

7.6 关键数据结构与不变量

结构	字段	不变量
有界缓冲区	`buf[]`, `read`, `write`, `count`	`0 ≤ count ≤ N`；`write = (read+count) mod N`
锁	`value`（FREE/BUSY）、等待队列	持有者唯一；只有持有者能 release
等待队列	线程链表	被唤醒的线程必须重新竞争锁（Mesa 语义）

7.7 经典陷阱

1. **用不同的锁保护同一份数据**：互斥失效（讲义在银行取款例子里强调“必须用同一把锁”）。
2. **临界区里做阻塞 I/O 或睡眠**：拉长临界区，破坏实时性；更糟的是可能死锁。
3. **忘记“检查条件”与“操作”必须同属一个临界区**：`if (count < N) enqueue()` 中间被打断就会越界。

4. **“概率很低”不等于“不会发生”**；单核上忙等还可能让持有锁的线程永远得不到 CPU（第 13 章的优先级反转）。

第 8 章 讲义 7–8：锁的实现（Implementing Locks）

8.1 动机：从“纸面算法”到“内核原语”

第 7 章证明：只用 load/store，正确性和效率都难以兼顾。于是要向下问：**硬件能给什么？** 讲义 7–8 给出的阶梯是：

```
只读写的软件算法（Bakery）
        │  太复杂、忙等
        ▼
关中断（仅单核、仅内核）
        │  用户不能用；多核代价高
        ▼
原子读改写指令（test&set / swap / CAS / LL-SC）
        │  忙等仍浪费 CPU
        ▼
test&set + 睡眠队列（内核锁）
        │  每次 release 都要进内核
        ▼
futex：用户态快路径 + 内核慢路径
```

8.2 单核方案：关中断

```
/* 朴素版：整个临界区关中断 —— 错误示范 */
void acquire(void) { disable_interrupts(); }
void release(void) { enable_interrupts(); }
```

问题（讲义逐条列）：

- **不能给用户用**：用户关中断后 `while(1);` 会让整机失去定时器与 I/O；
- **实时性**：临界区可能任意长，中断被延迟；
- **多核无效**：关一个核的中断，别的核照样进临界区（要靠核间消息，代价极高）。

正确做法是**只对“锁变量本身”做互斥**，把临界区留给用户：

```
int value = FREE;                  /* 锁变量 */
void acquire(void) {
    disable_interrupts();
    if (value == BUSY) {           /* 有人持有 */
        put_thread_on_wait_queue();
        sleep();                   /* 关键：睡眠与“检查+入队”必须原子 */
    } else {
        value = BUSY;
    }
    enable_interrupts();
}
void release(void) {
    disable_interrupts();
    if (wait_queue_not_empty()) {
        wake_one_thread();         /* 把等待者放回 ready 队列 */
    } else {
        value = FREE;
    }
    enable_interrupts();
}
```

这里有一个著名的坑：`sleep()` 与“恢复中断”的先后顺序。

顺序	后果
先使能中断，再入等待队列	release 可能在入队前发生 ⇒ 唤醒丢失 ⇒ 睡死
先入队，再使能中断，再 sleep	release 可能已经把该线程放回 ready 队列，但它仍以为自己持有锁 ⇒ 死锁
先入队，再 sleep，由调度器在切换到下一个线程时使能中断	正确（PintOS 的做法：`sema_down` 内部保证）

讲义把这条称为“meta-critical section 很短，但必须精确”的典型。

8.3 多核方案：原子读改写指令

指令	语义（伪代码）	架构
`test&set(&addr)`	返回旧值，并把 `addr` 置 1	多数架构
`swap(&addr, reg)`	交换寄存器与内存	x86 `xchg`
`compare&swap(&addr, old, new)`	若 `*addr == old` 则写 `new`，返回旧值	x86 `cmpxchg`、68000
`load-linked / store-conditional`	LL 读并监视，SC 仅在没有他人写过时成功	R4000、Alpha、ARM

```
/* 自旋锁：正确但浪费 CPU */
void acquire(int *lock) {
    while (test_and_set(lock))   /* 抢到返回 0，退出循环 */
        ;                        /* 自旋 */
}
void release(int *lock) { *lock = 0; }

/* 改进：test-and-test&set —— 先用普通读等，减少缓存行 ping-pong */
void acquire(int *lock) {
    while (*lock || test_and_set(lock))
        ;
}
```

讲义对自旋锁的评价：

- 优点：**不需要进内核**，用户态可用，多核可用；
- 缺点：**忙等浪费周期**，可能拖慢持有者；**优先级反转**（高优先级自旋者抢走低优先级持有者的 CPU）；
- 缓存一致性代价：每次 `test&set` 都是写，会让缓存行在核间来回弹（“ping-pong”），所以要用 test-and-test&set 的**只读自旋**。

8.4 两态锁与三态锁（futex 的核心思想）

```
/* 三态锁：UNLOCKED / LOCKED / CONTESTED */
typedef enum { UNLOCKED, LOCKED, CONTESTED } Lock;

void acquire(Lock *l) {
    if (compare_and_swap(l, UNLOCKED, LOCKED)) return;   /* 快路径：无竞争，零系统调用 */
    do {
        if (*l == UNLOCKED && compare_and_swap(l, UNLOCKED, CONTESTED)) return;
        futex_wait(l, CONTESTED);                        /* 慢路径：睡眠 */
    } while (1);
}
void release(Lock *l) {
    if (*l == CONTESTED) {
        *l = UNLOCKED;
        futex_wake(l, 1);                                /* 唤醒一个等待者 */
    } else {
        *l = UNLOCKED;                                   /* 无竞争者：不进内核 */
    }
}
```

要点：

- **uncontended 情况完全在用户态**（一次 CAS）；
- 有竞争时才陷入内核睡眠/唤醒；
- 仍有“release 必须看一眼是否有人等”的成本；进一步优化会引入“等待者计数”等技巧（讲义推荐 Ulrich Drepper 的 *Futexes are Tricky*）。

8.5 关键数据结构与不变量（PintOS 对应实现）

原语	实现要点
`lock`	`struct lock { struct thread *holder; struct semaphore sema; }`；`lock_acquire` 先 `sema_down`，成功者成为 holder
`sema`	`struct semaphore { unsigned value; struct list waiters; }`；`sema_down` 在 `value==0` 时把自己加入 `waiters` 并 `thread_block()`
关中断	`intr_disable()/intr_set_level()`；PintOS 单核，因此 `sema_down` 内部用关中断保护队列操作
原子性	单核 + 关中断即可；PintOS **不支持多核**（讲义 25 也提到这一点）

不变量：`lock->holder != NULL` 时 `sema.value == 0`；`sema_up` 必须在关中断下把等待者移入 ready 队列并 `thread_unblock`。

8.6 经典陷阱

1. **忘记 `release` 前的关中断**：release 与 acquire 之间存在窗口，`value` 被两个线程同时读到 FREE。
2. **在关中断状态下睡眠**：下一个线程可能永远不开中断（PintOS 断言 `intr_context()` 或直接挂死）；自旋锁不可用于长时间持有的临界区。
3. **单核自旋等待被换出的线程**永远等不到；`test&set` 返回**旧值**（0 表示抢到），别理解反。
4. **以为 futex 是系统调用号**：它是一族系统调用（wait/wake），核心是“用户态原子操作 + 内核等待队列”。

第 9 章 讲义 8–9：信号量（Semaphores）

9.1 动机：一把锁不够用

有界缓冲区有三条约束（互斥 + 两个调度约束）。用锁只能表达互斥；要表达“等缓冲区非空/非满”，需要一种**能计数、能让线程睡眠等待**的原语。Dijkstra 在 1960 年代末给出的答案就是**信号量**。

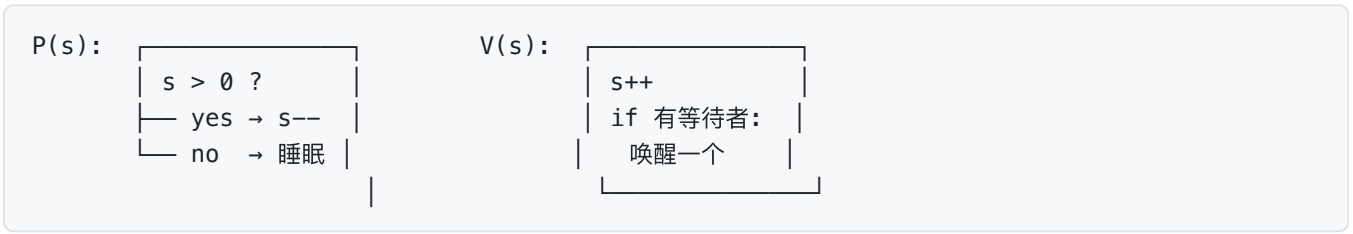
9.2 定义与语义

信号量 = 一个非负整数 + 两个原子操作：

操作	别名	语义
`P(s)` / `down(s)` / `wait(s)`	等待	等到 `s > 0`，然后 `s--`；否则睡眠
`V(s)` / `up(s)` / `signal(s)`	通知	`s++`；若有等待者，唤醒一个

关键性质（讲义逐条强调）：

- 值**不会为负**；
- 除初始化外**不能读/写**值（POSIX 允许读，但“技术上不属于正确接口”）；
- 操作必须原子；
- **P 中睡眠的线程不会错过 V 的唤醒**（即使两者同时发生）——这是“有历史”的语义，与条件变量相反。



9.3 两种典型用法

(a) 互斥（初值 1，binary semaphore / mutex）

```
semaphore mutex = 1;
P(&mutex);
/* critical section */
V(&mutex);
```

(b) 调度/顺序约束（初值 0）

```
semaphore ready = 0;
/* 线程 1 */            /* 线程 2 */
P(&ready);              V(&ready);   /* 线程 2 先完成某事，线程 1 才能继续 */
```

规则：**每条约束一个信号量**。有界缓冲区因此需要三个：

```
semaphore emptySlots = N;   /* 生产者等待：还有空位吗 */
semaphore fullSlots  = 0;   /* 消费者等待：还有可乐吗 */
semaphore mutex      = 1;   /* 队列互斥 */

void producer(item) {
    P(&emptySlots);         /* 先等资源，再拿锁！ */
    P(&mutex);
    enqueue(item);
    V(&mutex);
    V(&fullSlots);
}
void consumer(void) {
    P(&fullSlots);
    P(&mutex);
    item = dequeue();
    V(&mutex);
    V(&emptySlots);
}
```

9.4 顺序为什么重要（死锁的第一课）

顺序	结果
生产者 `P(empty)` → `P(mutex)`	正确
生产者 `P(mutex)` → `P(empty)`	若缓冲区满：持有 mutex 等空位；消费者要 mutex 才能取货 ⇒ **死锁**
V 的顺序	通常不影响正确性，但影响唤醒效率与公平性

讲义特别指出：**“P 的顺序颠倒会死锁”这件事并不直观**，这正是信号量“难以证明正确”的缺陷，也是引入监视器的动机。

9.5 用信号量实现其它原语

目标	做法
锁	初值 1 的信号量
条件等待（join）	初值 0；被等待者 `V`，等待者 `P`
计数型资源池	初值 = 资源数
读写锁（读优先）	`mutex` + `readers` 计数 + `roomEmpty` 信号量
屏障（barrier）	计数 + 两个信号量轮换

9.6 关键数据结构与不变量

```
struct semaphore {
    unsigned value;         /* 可用资源数，≥ 0 */
    struct list waiters;    /* 阻塞线程队列 */
};
```

不变量：

- `value > 0` ⇒ `waiters` 为空（实现可以选择不满足，但 PintOS 的风格是“计数 + 队列”）；
- `value == 0` 且 `waiters` 非空 ⇒ 每个等待者都在睡眠；
- `sema_up` 必须在关中断下完成“`value++` 或唤醒一个等待者”的原子选择，否则会丢失唤醒。

9.7 经典陷阱

1. **P/V 不配对**：多一个 P 会永久阻塞；多一个 V 会让互斥失效（两个线程同时进临界区）。
2. **用信号量当条件变量**：信号量**有历史**（V 先发生会被记住），条件变量**没有历史**（signal 时无人等待就丢失）。混用会导致“睡死”或“多吃”。
3. **在持有 mutex 时执行 P(资源)**：就是上面的死锁模式。
4. **忘记初始化**：`value` 是随机值，行为不可预测。
5. **`V` 的位置**：`V(mutex)` 应在离开临界区时立刻做，`V(fullSlots)` 放临界区外通常更好（减少唤醒后的竞争）。

第 10 章 讲义 9：监视器、条件变量与读者–写者（Monitors, CVs, RW）

10.1 动机：把“锁 + 等待”打包成语言级结构

信号量的两大问题：**一个原语承担两种职责**（互斥 + 调度），**顺序错误就死锁**。更干净的分解是：

- **锁**负责互斥；
- **条件变量（Condition Variable, CV）**负责“在临界区内等待某个条件成立”。

监视器（Monitor）= 一把锁 + 零个或多个条件变量。Java 的 `synchronized` 是语言级监视器；C/C++ 用 `pthread_mutex_t` + `pthread_cond_t` 手工拼装。

10.2 条件变量的三个操作

操作	语义	前提
`wait(cv, lock)`	**原子地**释放锁并睡眠；被唤醒时重新获取锁后返回	必须持有锁
`signal(cv)`	唤醒一个等待者（若有）	必须持有锁
`broadcast(cv)`	唤醒所有等待者	必须持有锁

与信号量的本质差别：

维度	信号量	条件变量
是否有“历史”	有（V 先发生会被记住）	无（signal 时无人等待则丢失）
能否在临界区内等待	不能（会死锁）	能（wait 会自动释放锁）
是否需要重新检查条件	不需要	**必须**（Mesa 语义）

10.3 Hoare vs Mesa 语义

```
Hoare（讲义：“most textbooks talk about”）：
  signaler 立刻把锁与 CPU 交给 waiter，自己阻塞
  ⇒ waiter 醒来时条件一定成立
  ⇒ 大量上下文切换，实现难

Mesa（讲义：“most real operating systems do this”）：
  signaler 把 waiter 放到 ready 队列，自己继续持有锁
  ⇒ waiter 稍后运行时条件可能已不成立
  ⇒ 必须用 while 循环重新检查条件
```

标准写法（**背下来**）：

```
lock_acquire(&m);
while (!condition)            /* 必须是 while，不能是 if */
    cond_wait(&cv, &m);
/* 此时条件成立，且持有锁 */
...
lock_release(&m);
```

10.4 有界缓冲区（监视器版）

```
void producer(item) {
    lock_acquire(&m);
    while (count == N) cond_wait(&notFull, &m);
    enqueue(item);
    cond_signal(&notEmpty);
    lock_release(&m);
}
void consumer(void) {
    lock_acquire(&m);
    while (count == 0) cond_wait(&notEmpty, &m);
    item = dequeue();
    cond_signal(&notFull);
    lock_release(&m);
}
```

注意：这里**不需要**单独的 mutex 信号量，因为监视器的锁已经保证互斥；也**不需要**记录“空/满”的计数信号量，因为 `count` 就是条件。

10.5 读者—写者（Readers—Writers）：状态机式推理

约束：

1. 无写者时，多个读者可同时访问；
2. 无读者且无写者时，一个写者可访问；
3. 同一时刻只有一个线程修改状态变量（由锁保证）。

状态变量：`AR`（活跃读者数）、`WR`（等待读者数）、`AW`（活跃写者数，0/1）、`WW`（等待写者数）。

```
/* Reader */
lock_acquire(&m);
while (AW + WW > 0) { WR++; cond_wait(&okToRead, &m); WR--; }
AR++;
lock_release(&m);
/* ... read ... */
lock_acquire(&m);
AR--;
if (AR == 0 && WW > 0) cond_signal(&okToWrite);
lock_release(&m);

/* Writer */
lock_acquire(&m);
while (AW + AR > 0) { WW++; cond_wait(&okToWrite, &m); WW--; }
AW++;
lock_release(&m);
/* ... write ... */
lock_acquire(&m);
AW--;
if (WW > 0) cond_signal(&okToWrite);
else if (WR > 0) cond_broadcast(&okToRead);
lock_release(&m);
```

讲义的逐步模拟（R1, R2, W1, R3 的顺序）值得自己手推一遍，重点看两件事：

- **写者优先**：新读者看到 `WW > 0` 就排队，避免写者饿死；
- **`broadcast` 而不是 `signal`**：读者可以批量放行，用 `signal` 只唤醒一个会浪费。

10.6 语言层面的坑

场景	风险	对策
C 的 `goto` 跳出临界区	漏掉 release	统一出口标签
`setjmp/longjmp`	非局部跳转跳过 release	禁止在临界区内使用
C++ 异常	异常路径漏 release	RAII（`std::lock_guard`）或 catch–release–rethrow
忘记 `while` 重检；Java 方法级 `synchronized` 导致意外死锁	条件已失效仍继续	永远 `while`；缩小同步块

10.7 关键数据结构与不变量

结构	字段	不变量
条件变量	等待线程队列	等待者一定**不持有**锁（wait 已释放）
监视器	一把锁 + 若干 CV	访问共享状态必须持锁；CV 操作必须持锁
RW 状态	`AR, WR, AW, WW`	`AW ≤ 1`；`AW>0 ⇒ AR==0`；计数非负

10.8 经典陷阱

1. **用 `if` 代替 `while`**：Mesa 语义下会被“虚假唤醒/抢跑”击穿。
2. **在不持锁时 signal**：C/POSIX 允许但语义危险；Java 会抛 `IllegalMonitorStateException`。
3. **用 signal 唤醒“可能不止一个”的等待者**：例如多个读者在等，`signal` 只放一个，剩下的一直等（应 `broadcast`）。
4. **读者优先导致写者饿死**：讲义专门问 “Can readers starve?”——会，若不限制新读者。
5. **把 CV 当成“事件计数器”**：CV 无历史，事件必须先改变状态变量，再 signal。

第 11 章 讲义 10：调度 1 —— 目标与经典策略（Scheduling 1）

11.1 动机：机制已备，策略待定

讲义 10 把前半学期的“机制”收束成一句话：上下文切换已经能工作，现在要回答**“下一个该跑谁”**。这就是**策略（policy）**，而切换本身是**机制（mechanism）**。

11.2 四个目标，彼此冲突

目标	定义	谁在乎
最小化响应时间	从提交到完成/首次响应的时间	交互用户
最大化吞吐	单位时间完成的任务数	批处理、服务器
公平	每个参与者得到应得的份额	多用户系统
可预测性	最坏情况可界定	实时系统

讲义的关键洞察：**“公平”与“最小平均响应时间”是冲突的**——“Better average response time by making system less fair”。另一个常被忽略的点：**公平的对象是谁**？用户还是程序？如果我跑 1 个编译任务、你跑 5 个，按程序分你拿 5 倍 CPU，这算公平吗？

11.3 工作负载模型

- 程序是 **CPU burst 与 I/O burst 交替**的序列；
- 调度决策发生在 burst 边界（或时间片到期）；
- **所有模型都建立在简化假设上**（一个用户一个程序、一个程序一个线程、程序互相独立），真实系统并不满足——所以要理解“模型何时失效”。

11.4 FCFS / FIFO

```
P1(24) P2(3) P3(3)   到达顺序 P1,P2,P3
├──────────────────────┼──┼──┤
0                      24  27  30
平均等待 = (0+24+27)/3 = 17
```

- 优点：实现极简、无饥饿；缺点：**护航效应（convoy effect）**——短任务排在长任务后面（“在超市只买牛奶却排在装满购物车的顾客后面”）；若顺序变成 P2,P3,P1，平均等待降到 3。

11.5 Round Robin（RR）

- 每个线程获得最多 `q` 的时间片，用完被抢占并排到队尾；
- `n` 个就绪线程、时间片 `q`：每人最多等 `(n-1)q`；
- `q` 太小 ⇒ 上下文切换开销占比爆炸；`q` 太大 ⇒ 退化成 FCFS；
- 现代典型：`q` = 10–100 ms，切换开销 0.1–1 ms（约 1%）；
- 讲义的反直觉结论：**即使切换零成本，RR 的总完成时间也可能比 FCFS 差**（10 个等长任务时，RR 的平均完成时间远差于 FCFS），因为 RR 让所有任务“同时慢”。

11.6 优先级调度

```
就绪队列按优先级组织：
 priority 3 : [Job1] [Job2] [Job3]
 priority 2 : [Job5] [Job6]
 priority 1 : [Job7]
总是取最高优先级的队首
```

- 优点：能表达“谁更重要”；
- 缺点：**饥饿**（低优先级永远轮不到）、**优先级反转**（第 13 章）；
- 需要配合 **老化（aging）** 或 **动态优先级**。

11.7 关键数据结构与策略对比

策略	抢占	优点	缺点	starvation
FCFS	否	简单、公平（按到达）	护航效应	无
RR	是	响应好、公平	切换开销；总完成时间可能差	无
静态优先级	可选	表达重要性	低优先级饿死	有
SJF/SRTF	SRTF 是	平均响应最优	需预测未来	长任务饿死
MLFQ	是	近似 SRTF、无需预知	参数多、可被“骗”	可能
比例份额（lottery/stride/CFS）	是	按权重分配、无饥饿	需调权重	无

11.8 经典陷阱

1. **把 RR 的时间片设得过小**：切换开销吞掉吞吐；过大会退化成 FCFS。
2. **静态优先级不做老化**：低优先级任务饿死。

第 12 章 讲义 11：调度 2 —— SJF/SRTF、MLFQ 与工业调度器

12.1 动机与机制：如果知道未来（SJF / SRTF）

- **SJF（Shortest Job First）**：非抢占，选剩余计算量最小者；
- **SRTF（Shortest Remaining Time First）**：抢占版，新任务更短就切换；
- **理论结果**：在最小化平均响应时间上，SJF 在非抢占中最优，SRTF 在抢占中最优（讲义说“Provably optimal”）；
- 退化情形：所有任务等长 ⇒ SJF 与 FCFS 相同（此时 FCFS 就是最优）；
- 致命问题：**无法预知未来**，且长任务会被饿死。

12.2 用过去预测未来：指数平均

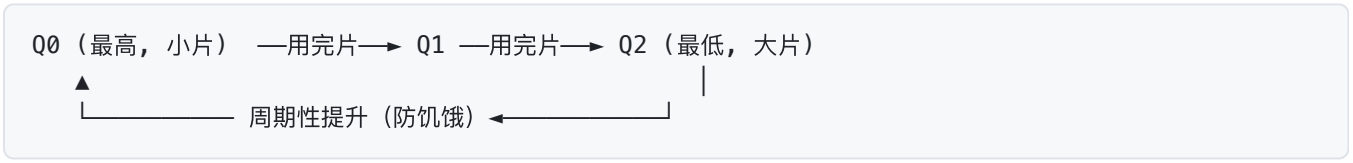
```
τ(n+1) = α · t(n) + (1−α) · τ(n)
   t(n)   = 第 n 次 burst 的实测长度
   τ(n)   = 第 n 次的预测值
   α      = 0..1，越大越“看重最近一次”
```

依据是**程序行为的可预测性**（I/O 密集型过去如此，将来大概率如此）。若程序行为随机，这个方法就无效——“这也是为什么 OS 里到处是‘启发式’”。

12.3 MLFQ（Multi-Level Feedback Queue）

规则（讲义版）：

1. 任务从最高优先级队列进入；
2. 时间片用完仍未完成 ⇒ **降一级**；
3. 时间片内主动放弃 CPU（I/O、等待）⇒ **保持或升一级**；
4. 队列之间：固定优先级（高优先级全跑完才跑低的）或按比例分配时间。



- 效果：**CPU 密集型任务快速下沉，I/O 密集型任务留在高层**，近似 SRTF 而不需要预知；
- 风险：**游戏（gaming）**——故意插入无意义 I/O 保持高优先级（讲义举 Othello 程序“加 printf 反而更快”的例子）；
- 现实变体：BSD 调度器、Solaris、Windows 都用多级反馈思想。

12.4 工业案例：O(1) 调度器（Linux 2.6 早期）

- 140 个优先级：0–99 给实时（值越小优先级越高），100–139 给普通任务（nice −20..19 映射）；
- 每个优先级一个队列，**140 位位图**记录哪些优先级非空 ⇒ 查最高优先级是 `ffs` 一条指令，O(1)；
- **active / expired 两个队列**：时间片用完进 expired，全部 active 用完后交换；
- 时间片由优先级线性映射；普通任务的优先级根据 `sleep_avg` 动态 ±5（交互型奖励）；
- 问题：启发式太多、交互判定可被欺骗。

12.5 CFS（Completely Fair Scheduler）

核心思想：**用“虚拟运行时间（vruntime）”记账**，总是选 vruntime 最小的线程。

```
vruntime += 实际运行时间 × (NICE_0_WEIGHT / weight)
weight = 1024 / (1.25 ^ nice)          /* nice 越小权重越大 */
```

参数	含义	讲义给的例子
Target Latency	所有可运行线程轮一遍的时间	20 ms
Minimum Granularity	单个时间片下限	1 ms
时间片	`Q = Target_Latency × weight_i / Σweight`	2 线程权重 1:4 ⇒ 4ms / 16ms
n 过大	`Q` 被压到 Minimum Granularity	200 进程 ⇒ 每片 1ms

实现：**红黑树**按 vruntime 排序，`O(log N)` 插入/删除，缓存最左节点；睡眠线程的 vruntime 不前进（自动获得交互性）。

12.6 关键数据结构与调度器对比（含 PintOS 的落点）

调度器	决策依据	复杂度	对应讲义/项目
FCFS	到达顺序	O(1)	讲义 10
RR	轮转 + 时间片	O(1)	讲义 10
严格优先级	priority 数值	O(n) 或 O(1)（位图）	**Project 2 要求**
MLFQ	多级队列 + 反馈	O(1)	讲义 11
O(1)	位图 + active/expired	O(1)	讲义 11
CFS	vruntime 最小	O(log N)	讲义 11/12

PintOS 的 `thread_tick` + `intr_yield_on_return` 就是“时间片到期强制切换”的机制；Project 2 要求在此基础上实现严格优先级（高优先级就绪就抢占）。

12.7 经典陷阱

1. **把 MLFQ 的“降级”写成“永久降级”**：必须有周期性提升，否则长任务饿死。
2. **忘记“时间片按优先级缩放”**：高优先级任务应更频繁、未必更长。
3. **在 CFS 中直接比较真实运行时间**：必须比较 vruntime，否则权重无效。

4. **以为 SJF 可实际实现**：除非能准确预测 burst，否则只能近似（MLFQ/CFS）；调度决策在临界区内，队列操作必须 O(1) 或 O(log N)。

第 13 章 讲义 12：饥饿、优先级反转与比例份额调度

13.1 动机：三个容易混淆的概念

概念	定义	能否自行恢复
饥饿（Starvation）	线程无限期无法推进	可能（条件变化后能恢复）
死锁（Deadlock）	循环等待，无法自行恢复	不能（需外部干预）
活锁（Livelock）	线程在动但整体无进展	可能（取决于随机/退避）

讲义的原话：**“Deadlock ⇒ Starvation，但反之不成立”**。

工作守恒（work-conserving）调度器：只要有人可运行就不让 CPU 空闲。非工作守恒会**人为制造饥饿**（但有时是必要的，例如实时系统留出余量）。

13.2 优先级反转（Priority Inversion）

```
优先级：  H（高）  >  M（中）  >  L（低）

L 持有锁 ──► H 请求锁，阻塞
             M 就绪（不碰锁）──► 调度器选 M
             M 一直跑 ──► H 永远等不到（被“中等优先级”间接压制）
```

三种解法：

方案	做法	代价
优先级继承（inheritance）	持有者临时继承等待者的优先级	需追踪“谁在等谁”；释放时恢复
优先级天花板（ceiling）	锁的优先级 = 可能使用它的最高优先级；持有即提升	需静态分析
禁止抢占临界区	持有锁期间不被抢占	影响响应时间

13.3 Pathfinder（1997 火星探路者）

讲义用它做“真实事故”案例：

- 火星车反复重启、丢失数据；

- 原因：**优先级反转**——低优先级的 ASI/MET 采集任务持有 mutex，高优先级的数据分发任务等它，中优先级的通信任务不停抢占；
- 更糟的是：开发者**为性能关掉了优先级捐赠**（priority inheritance）；
- 修复：远程打开优先级捐赠并上传补丁。

这段故事的工程教训：**“为性能关掉正确性机制”是最危险的优化**。

13.4 比例份额：Lottery 与 Stride

Lottery scheduling：

- 每个任务持有若干彩票；每个时间片随机抽一张，中奖者运行；
- 期望 CPU 份额 = 票数 / 总票数；
- 优点：负载变化时行为平滑；每任务至少给 1 张票 ⇒ **无饥饿**；
- 缺点：随机性导致短期波动；票数分配是策略问题。

短任务数 / 长任务数	每个短任务份额	每个长任务份额
1 / 1	91%	9%
0 / 2	—	50%
2 / 0	50%	—
10 / 1	9.9%	0.99%
1 / 10	50%	5%

（表中数字为讲义“短任务 10 票、长任务 1 票”的例子。）

Stride scheduling：确定性的比例份额。

```
stride_i = K / tickets_i          /* 票越多，步长越小 */
每次选 pass 值最小的任务运行，运行后 pass += stride
```

例：总票 400，A=100、B=50、C=250（K=10000）⇒ stride A=100、B=200、C=40；C 最频繁。

13.5 多核与亲和性

每核一个就绪队列 + 后台负载均衡；**亲和性（affinity）**把线程放回上次的核以复用 cache/分支预测状态；自旋锁每次 `test&set` 都是写，会让缓存行在核间 ping-pong，故用 test-and-test&set；SMT 让逻辑核数大于物理核。

13.6 关键数据结构与不变量

结构	字段	不变量
优先级调度器	每线程 `base_priority` + 捐赠链	有效优先级 = max(base, 所有捐赠)
捐赠记录	`(donor, lock)`	释放锁时必须删除对应记录并重算
lottery	每任务票数	总票数 > 0；每任务至少 1 票
stride	`pass`、`stride`	选 pass 最小者；运行后 `pass += stride`

13.7 经典陷阱

1. **把“优先级高”当成“不会饿”**：静态优先级 + 持续高负载 ⇒ 低优先级饿死。
2. **只在锁上做优先级继承，忘了信号量/CV**：PintOS Project 2 明确只要求 lock 做捐赠，但**三类原语都要按优先级唤醒**。
3. **用 lottery 做硬实时**：概率性保证不满足 deadline。
4. **忽略 stride 的计数器回绕**：需要用 64 位或定期归一化。
5. **在多核上让同一线程在两个核同时运行**：必须保证线程只在一个核上（PintOS 单核，天然满足）。

第 14 章 讲义 13：实时调度与死锁（Realtime & Deadlock）

14.1 动机：实时 ≠ 快

讲义反复强调：**real-time 是关于“可预测性”，不是“算得快”**。

类型	目标	典型算法
硬实时（hard）	所有 deadline 必须满足（安全攸关）	RMS、EDF、LLF
软实时（soft）	以高概率满足	CBS、优先级 + 预留带宽

14.2 周期任务模型

任务 `T_i = (P_i, C_i)`：周期 `P_i`、每周期计算量 `C_i`；到达时刻 `A_{i,k+1} = A_{i,k} + P_i`；deadline `D_i = A_i + P_i`；利用率 `U_i = C_i / P_i`。

RMS（Rate Monotonic Scheduling）：周期越短优先级越高（静态优先级）；可调度性充分条件：

$$\Sigma U_i \le n(2^{1/n} - 1) \quad \to \quad \ln 2 \approx 0.693 \ (n \to \infty)$$

讲义给的失败例子：$T1=(4,1)$、$T2=(5,2)$、$T3=(7,2)$，$\Sigma U \approx 0.936 > 3(2^{1/3}-1) \approx 0.780$，**不满足判据**，且实际排程确实错过 deadline。注意：判据是**充分不必要**——不满足不代表一定失败，但本例确实失败。

EDF（Earliest Deadline First）：动态优先级，总是运行**绝对 deadline 最近**的任务；可调度性判据（$D_i = P_i$ 时）是**充要**的：

$$\Sigma U_i \le 1$$

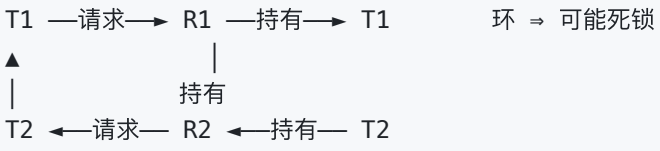
EDF 在“可行性”和“最小化最大延迟”意义上最优（Liu & Layland, 1973）。代价：需要知道 deadline、实现更复杂、过载时行为“崩得突然”（ domino effect）。

14.3 死锁的四个必要条件（必背）

条件	含义	打破方法
互斥（Mutual Exclusion）	资源一次只能给一个线程	共享化（只读、spooling）
持有并等待（Hold and Wait）	持有资源同时等新资源	一次性申请全部；释放后再申请
不可抢占（No Preemption）	资源只能自愿释放	允许抢占/回滚
循环等待（Circular Wait）	存在等待环 T1→R1→T2→…→T1	**全局加锁顺序**

只要破坏任意一个，就不会死锁。工程上最实用的是第四条：**给所有锁定义全序**（例如按地址/编号升序获取）。

14.4 资源分配图与处理策略



策略	做法	代表
预防（prevention）	破坏四条件之一	加锁顺序、一次性申请
避免（avoidance）	动态检查“是否进入不安全状态”	**银行家算法**
检测 + 恢复	允许发生，定期检测环，杀线程/回滚	数据库事务

策略	做法	代表
忽略	“鸵鸟算法”	Unix/Linux 的普遍做法

安全状态：存在一个顺序 `{T1..Tn}`，使每个 `Ti` 的剩余请求都能被“当前空闲 + 前面线程释放”满足。银行家算法在每次请求时**假装授予**，然后跑安全性检查；不安全就等待。

14.5 Dining Philosophers / 哲学家与筷子

- 5 人 5 筷，每人需两筷；
- 朴素实现（先左后右）⇒ 可能全员各持一筷等另一支 ⇒ 死锁；
- 解法：**给筷子编号，按序取**（破坏循环等待）；或“只允许 4 人同时上桌”（破坏持有并等待）；或服务员（集中仲裁）。

14.6 关键数据结构与不变量

结构	用途	不变量
资源分配图	检测死锁	有环 ⇒ 可能死锁（单实例资源时“有环 ⇔ 死锁”）
`Available[m]`、`Max[n][m]`、`Allocation`、`Need`	银行家算法	`Need = Max − Allocation`；`Available ≥ 0`
周期任务表	RMS/EDF	`Σ C_i/P_i ≤ 1`（EDF 充要）

14.7 经典陷阱

1. **以为 RMS 判据是必要条件**：它是充分条件；不满足仍可能可调度。
2. **把 EDF 当成“永远正确”**：过载时 EDF 会让所有任务一起错过 deadline（domino effect），RMS 则“稳定地错过低优先级任务”。
3. **忘记“锁顺序”要全局统一**：只在某个模块内排序，跨模块仍可能成环。
4. **在持锁时调用可能获取另一把锁的函数**（回调、`printf`、内存分配）：隐式嵌套锁是死锁温床。
5. **用“超时重试”当死锁解决方案**：可能活锁，且破坏实时性。

第 15 章 讲义 14：虚拟内存 1 —— 地址空间、分段与页表

15.1 动机：为什么不能“直接给物理地址”

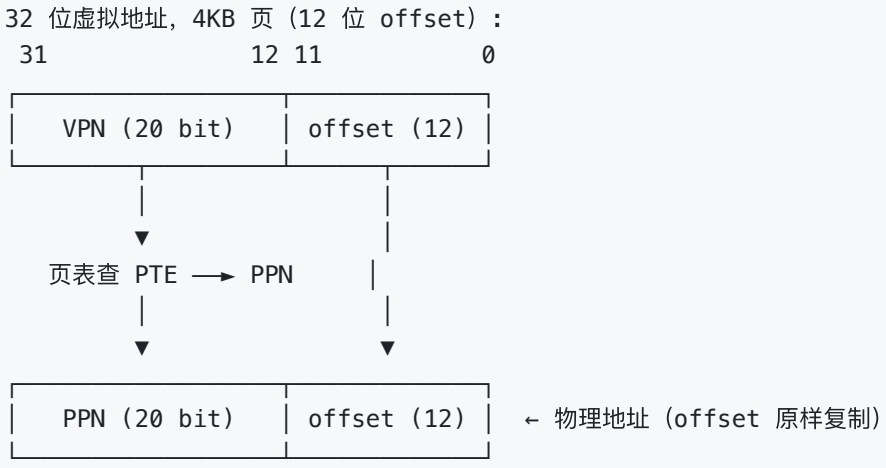
- 进程的完整状态（代码、数据、栈、堆、页表）都在内存里，**不能让两个线程/进程用同一块物理内存**；
- 但也不能让进程直接看到物理地址，否则它可以读写别人；
- OS 又**不可能逐条指令检查每次访存**（太慢）。

结论：把“检查”做进硬件，让**常见情况快、罕见情况陷入内核**——这就是地址翻译 + 缺页异常的分工。

15.2 从连续到非连续：三种方案的对比

方案	虚拟→物理	优点	缺点
Loader/Linker 重定位	加载时改写地址	无硬件成本	无法保护；位置固定；不能共享
Base & Bound	`PA = VA + Base`，检查 `VA < Bound`	一次加法；简单	单段连续；外部碎片；难共享
分段	段表：Base + Limit	支持多段（code/data/stack）	外部碎片；整段换入换出
分页	页表：VPN → PPN	无外部碎片；易共享；按需调页	页表可能巨大；内部碎片；TLB 开销

15.3 分页的地址拆分



- 页表项（PTE）至少含：**valid、read/write/execute 权限、user/supervisor、accessed、dirty、PPN**；
- 简单页表 = 一个以 VPN 为下标的数组 ⇒ **稀疏地址空间下浪费巨大**：32 位、4KB 页、进程只用低端 code 与高端 stack，中间上百万个 PTE 全是“无效”。

15.4 段页式（x86 的真实做法）

```
逻辑地址 [ 段选择子 | 偏移 ]
              │
              ▼
       段描述符（Base, Limit, DPL）
              │  Base + 偏移
              ▼
       线性地址（32 位）──► 两级页表 ──► 物理地址
```

- x86 有 6 个段寄存器（CS/SS/DS/ES/FS/GS），`CS.RPL` 即当前特权级 CPL；
- 现代 32/64 位系统把段“压平”（base=0, limit=4G），只用 **FS/GS 指向线程本地存储（TLS）**；
- 讲义强调：**分段在 x86 上不能真正关闭**，只是被设置成扁平。

15.5 共享、写时复制、ASLR

技术	机制	作用
共享代码段	多个进程的 PTE 指向同一物理页，只读	节省内存
共享库 / mmap	文件页映射进地址空间	按需加载、进程间共享
写时复制（COW）	`fork` 后父子 PTE 都只读，写时缺页再复制	廉价 fork
ASLR	随机化 code/stack/heap 基址	增加攻击难度（讲义：需要位置无关代码）
内核页表隔离（KPTI）	用户态不映射内核	缓解 Meltdown（讲义 16/18 提到）

15.6 关键数据结构与不变量

结构	内容	不变量
页表基址寄存器（CR3）	当前进程页目录的物理地址	切进程必须换；用户态不可写
PTE	PPN + 权限 + 状态位	无效 PTE 的其余位可被 OS 复用（如 swap 位置）
`struct thread` / `struct process`	持有页目录指针	线程切换不换 CR3；进程切换必须换

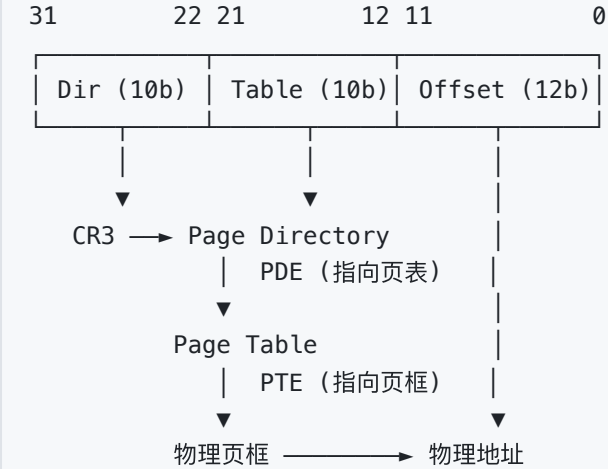
15.7 经典陷阱

1. **忘记“offset 不翻译”**：页大小决定 offset 位数，算错就全错。
2. **以为页表在虚拟内存里就“不用管”**：页表页本身也要占物理内存，且必须常驻（或被递归映射）。
3. **在用户态可写页表页**：等于把整机交给用户。
4. **权限位是“与”关系**：用户态访问需 PTE.user=1 且 CPL=3；另外分段并未“死去”，x86 仍用 FS/GS 做 TLS。

第 16 章 讲义 15：多级页表、TLB 与缓存

16.1 动机与机制：把“大数组”变成“树”（两级页表）

x86 32 位的经典拆分 **10–10–12**：

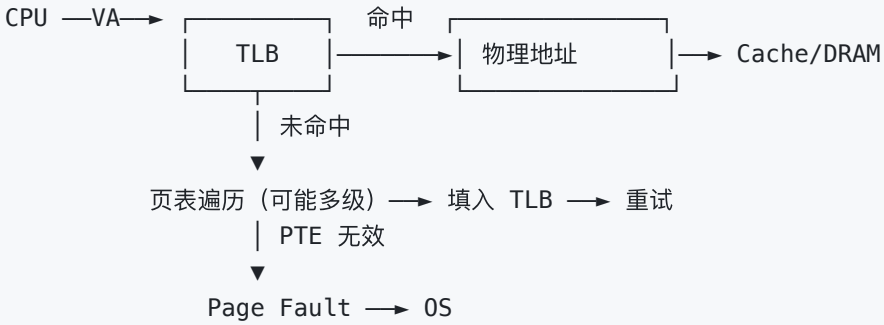


- 每级表 1024 项 × 4B = 4KB，正好一页；
- **稀疏地址空间只需分配“用到的”二级表**：一个进程可能只用 3–4 个二级表，而不是 1024 个；
- 代价：一次访存要 **3 次内存访问**（PDE、PTE、数据），所以必须有 TLB。

方案	表大小	稀疏支持	查找次数
单级页表	4MB（2^20 × 4B）	差	1
两级页表	4KB 目录 + 按需二级表	好	2
三级/四级（x86-64）	更多级	更好	3–4
反置页表（inverted）	∝ 物理页数	与虚拟空间无关	需 hash 查找

反置页表（讲义 15）：以物理页框为索引，记录“哪个 (pid, vpn) 映射到我”；优点是表大小只与物理内存有关（对 64 位地址空间极有吸引力，PowerPC/UltraSPARC/IA64 用过），缺点是 hash 冲突链复杂、缓存局部性差、共享页难处理。

16.2 TLB：翻译的缓存



要点：

- TLB 很小（几十到几千项），**通常全相联**（冲突代价太高）；可用“TLB slice”（小 direct-mapped 前置）降低功耗/延迟；
- 表项含 **ASID/PCID** 时可跨进程保留，否则切进程要 flush；
- 命中的是“端到端翻译结果”，与页表级数无关；
- **TLB 一致性**：OS 改 PTE（换入/换出、改权限、unmap）后必须 `invlpg` 或 flush，否则硬件仍用旧翻译。

16.3 缓存组织：为什么 TLB 与 cache 的设计互相牵制

类型	索引/标记	优点	缺点
Physically Indexed, Physically Tagged（PIPT）	物理地址	无别名问题；切进程不用 flush	必须在 TLB 之后，处在关键路径
Virtually Indexed, Physically Tagged（VIPT）	虚拟索引 + 物理标记	可与 TLB 并行查找	可能别名（同一物理页映射到多个虚拟地址）
Virtually Indexed, Virtually Tagged（VIVT）	全虚拟	最快	切进程必须 flush；别名问题严重

讲义给的实例（某 x86 核）：L1 I-Cache 32KB/8 路、L2 2MB、L3 共享；L1 DTLB 64 项 4 路（4KB 页）、L2 STLB 1536 项 12 路（4KB + 2MB 页）。这解释了为什么“大页（2MB/1GB）”重要：**一个 TLB 项覆盖更大范围**。

16.4 缓存缺失四分类（讲义 15/16 反复用）

类型	原因	对策
Compulsory（冷启动）	第一次访问	预取
Capacity（容量）	工作集大于缓存	增大缓存
Conflict（冲突）	映射到同一组	提高相联度 / 改索引
Coherence（一致性）	其它核/设备修改	一致性协议、invalidate

在虚拟内存里没有 conflict miss（全相联），但有 **policy miss**：页本来在内存里，却被错误地换出。

16.5 关键数据结构与不变量

结构	字段	不变量
TLB 项	VPN、PPN、权限、ASID、valid	与页表一致（OS 负责维护）
PDE/PTE	PPN、P/W/U、A/D	无效项的剩余位可存 swap 信息
cache line	tag、index、offset、valid、dirty	写回策略下 dirty 位必须准确

16.6 经典陷阱

1. **改了 PTE 却不失效 TLB**：最典型的“偶发、难复现”bug（换页后仍访问旧页框）。
2. **认为“TLB 命中率高就没事”**：页表遍历虽少，但一旦 miss 代价是几十到上百周期。
3. **在 VIVT 缓存上切进程不 flush**：不同进程的同一虚拟地址会串数据。
4. **大页放大内部碎片**（2MB 页装 4KB 数据浪费 512 倍）；cache 的写回与页的 dirty 位虽分层不同，思想同源（延迟写 + 脏标记）。

第 17 章 讲义 16：TLB 一致性、精确异常与请求调页

17.1 动机与机制：缺页的完整路径

```
进程访问 VA
   │
   ▼
MMU 查 TLB ──命中──► 访问物理内存（可能再走 cache）
   │未命中
   ▼
查页表 ──有效──► 填 TLB，重试指令
   │无效 / 权限不符
   ▼
Page Fault（同步异常，压入错误码与 PC）
   │
   ▼
OS 的 page_fault_handler:
   1. 判断 fault 类型：未映射 / 权限 / 未在内存 / 越界
   2. 非法访问 ⇒ 终止进程（SIGSEGV）
   3. 合法但缺页 ⇒ 选 victim（必要时写回）、读入页、改 PTE、失效 TLB
   4. 返回，硬件重试**同一条指令**
```

17.2 精确异常（precise exception）

要能“重试同一条指令”，硬件必须保证：

- 保存**出错指令的 PC**（不是下一条）；
- 保存足够的处理器状态，使指令可以重新执行；
- 之前的指令全部完成，之后的指令全部未开始（像没发生）。

难点：

情况	问题	处理
指令有副作用（`mov (sp)+, 10`、`strcpy`）	重试会重复副作用	要么回滚，要么“完成剩余部分”
延迟分支	需要 PC 与 nPC	硬件保存两个 PC
流水线/乱序执行	精确异常代价高	x86 选择精确；部分架构用 imprecise + 软件修复

17.3 请求调页（Demand Paging）

把 DRAM 当作磁盘的 cache：

cache 概念	虚拟内存对应
block size	1 页（如 4KB）
组织	**全相联**（任意虚拟页 → 任意物理框）

cache 概念	虚拟内存对应
定位	先 TLB，再页表
替换策略	近似 LRU（Clock 等）
写策略	**写回**（必须 dirty 位）
miss 代价	磁盘 I/O（毫秒级）

有效访问时间（EAT）：

```
EAT = (1 − p) × 内存访问时间 + p × 缺页服务时间
    = 200ns + p × 8ms
p = 1/1000  ⇒ EAT = 8.2μs（慢 40 倍）
要求“ slowdown < 10% ” ⇒ p < 2.5×10^−6，即约 40 万次访问才有一次缺页
```

这个数字说明：**缺页率必须极低**，否则任何优化都白费。

17.4 `fork` 与 COW、按需加载、mmap

场景	做法
`fork`	复制页表，父子 PTE 都设为只读；写时缺页再复制页（COW）
`exec`	丢弃旧地址空间，新建页表，只映射代码/数据的**文件页**，按需缺页读入
`mmap`	把文件页映射进地址空间；缺页时从文件读，`msync` 写回
栈增长	访问栈下方“guard page” ⇒ 缺页 ⇒ OS 扩栈（或报 SIGSEGV）
`sbrk`	扩大堆，只改“堆顶”记录，物理页按需分配

17.5 缺页处理中的“自由页框从哪来”

- 内核维护 **free list**，后台线程（pageout daemon/reaper）先回收干净的久未使用页，再写回脏页回收，最后才 swap out 整个进程；**缺页处理本身可能缺页**（页表页不在内存），需预留内核页或锁住页表页。

17.6 关键数据结构、不变量与经典陷阱

不变量：

- 一个物理页框在同一时刻只被一个 (进程, 虚拟页) 独占（共享页除外，需引用计数）；
- PTE 有效 ⇒ 对应页框在内存且引用计数 ⩾ 1；
- 缺页处理完成后，PTE 与 TLB 必须一致。

陷阱：

1. **在缺页处理中睡眠却不允许阻塞**：中断上下文不能睡眠；缺页处理运行在进程上下文，可以睡眠（这正是“按需调页”可行的原因）。
2. **忘记 `dirty` 位就丢弃页**：数据丢失。
3. **COW 的引用计数错**：父进程退出时把共享页释放，子进程访问即崩溃。
4. **栈溢出与 guard page 混淆**（无限递归应报错而非无限扩栈）；请求调页下缺页是**正常路径**，只有非法访问才是错误。

第 18 章 讲义 17：页面置换与工作集（Page Replacement）

18.1 动机：为什么置换策略重要

物理内存有限，缺页时必须选一个 victim。选错的代价是**毫秒级磁盘 I/O**；选对了可以把缺页率压到 10^−6。理想算法是 **MIN（Belady 最优）**：替换“未来最远才被使用”的页——但需要预知未来，只能作为上界。

18.2 四种基本算法

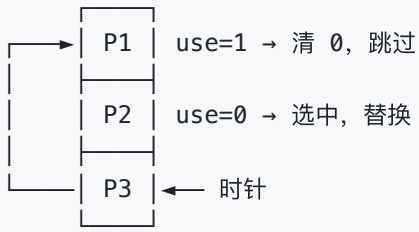
以引用串 `A B C A B D A D B C B`、3 个页框为例：

算法	规则	缺页数	特点
FIFO	替换最早进入的页	7	简单；会把热页踢出；**有 Bélády 异常**
MIN/OPT	替换未来最远使用的页	5	不可实现，作为下界
LRU	替换最久未使用的页	5	接近最优；实现代价高（每次访问都要更新）
Clock（二次机会）	环形扫描，use=1 则清零放过，use=0 则替换	6—7	LRU 的廉价近似

Bélády 异常：FIFO 下增加页框数反而可能增加缺页（引用串 `A B C D A B C D A B C D` 在 3 框时 9 次缺页，4 框时 10 次）。LRU/MIN 是**栈算法（stack algorithm）**，满足“M 框的驻留集 ⊆ M+1 框的驻留集”，因此不会出现该异常。

18.3 近似 LRU 的三种工程方案

(a) Clock / Second Chance



- 硬件在每次访问时置 **use/accessed 位**；OS 扫描时读并清零；
- 若所有位都为 1，最坏扫一圈后必有 0（因为扫描过程中被清零）；
- 时针快 ⇒ 缺页多或引用位被大量置位；时针慢 ⇒ 空闲内存多。

(b) Nth Chance：给每页 N 次机会，计数到 N 才替换；N 大 ⇒ 更接近 LRU，但找 victim 更慢。

(c) Second-Chance List（VAX 风格）：把内存分成 **Active list（可读写）** 与 **SC list（无效 PTE）**；访问 Active 全速；溢出页移到 SC 头部；命中 SC 页则移回 Active；缺页时从 SC 尾部取 victim。好处是缺页处理**不需要扫描**，坏处是共享页处理复杂。

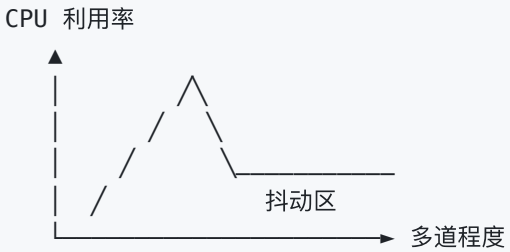
18.4 没有硬件 use/dirty 位怎么办（软件模拟）

- **模拟 dirty**：把所有可写页在 PTE 里标为只读；写触发缺页 ⇒ OS 记“脏”，再改为可写；
- **模拟 use**：类似地用“只读 + 缺页”捕获读；或按周期把 PTE 置无效，缺页即“使用过”；
- 讲义举 VAX 为例：架构师问 OS 开发者“需不需要 use 位”，回答“不需要”，于是没实现，后来被证明是个麻烦（但 VAX 仍然成功）。

18.5 全局 vs 局部、分配策略、抖动

策略	含义	后果
全局置换	从所有页框里选 victim	吞吐高；一个进程可能抢别人的页
局部置换	只从本进程的页框里选	隔离好；可能“抱着没用的页饿死”
等分分配	每进程 m/n 框	简单；不适合大小差异大的进程
比例分配	按进程大小分配	更合理
按缺页率调整（PFF）	缺页率高 ⇒ 加框，低 ⇒ 减框	自适应

抖动（Thrashing）：



- 现象：缺页率极高，CPU 大量时间在等 I/O，利用率反而下降；
- 根因：**所有进程的工作集之和 > 物理页框数**；
- 对策：**工作集模型**——用最近 Δ 次访问的页集合 `WS_i` 估计需求 `D = Σ|WS_i|`；若 `D > m`，**挂起/换出**某些进程（这是“降级多道程度”的反直觉操作）。

18.6 关键数据结构与不变量

结构	用途	不变量
页框描述符数组	记录每框的 (进程, vpn)、use、dirty、引用计数	总框数固定
free list	可用框	缺页时优先取；取空则触发回收
每进程驻留集	当前映射的页	与页表一致
swap 区	换出的页	每页有唯一 swap 槽位

18.7 经典陷阱

1. **用 FIFO 做 VM 置换**：可能踢掉正在用的页，还可能出现 Bélády 异常。
2. **以为“LRU 一定最好”**：LRU 对**循环扫描**型访问（比工作集大的顺序循环）会 100% 缺页，MIN 反而更好。
3. **忘记脏页回写**：直接丢弃 dirty 页 = 数据损坏。
4. **抖动时继续增加多道程度**（更糟）；共享页置换必须等引用计数归零；忘记脏页回写会丢数据。

第 19 章 讲义 18：通用 I/O 与设备驱动（General I/O, Device Drivers）

19.1 动机：12 个数量级的差异

设备的速率从“每秒几个字节”到“每秒 GB”，形态从键盘到 GPU。OS 要在**不改变上层接口**的前提下支持它们——这就是“抽象”的胜利。

19.2 硬件路径

```
CPU ──► 总线（PCIe 等）──► 控制器（Controller）──► 设备
                                  │
                                  ├─ 控制寄存器（命令/状态）
                                  ├─ 数据寄存器 / FIFO
                                  └─ DMA 引擎（直接读写内存）
```

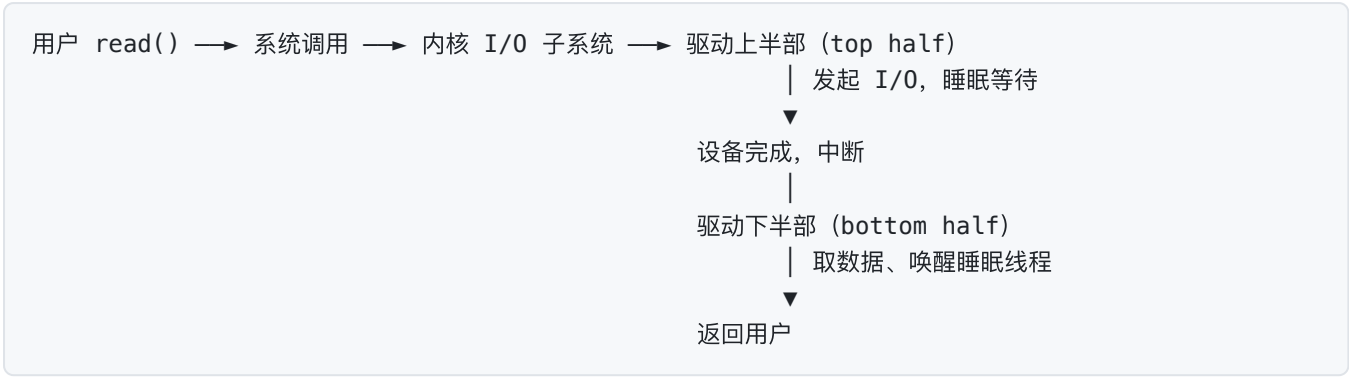
访问方式	指令	例子	特点
Port-mapped I/O	`in` / `out`	x86 `out 0x21, al`	独立 I/O 空间；需特权
Memory-mapped I/O（MMIO）	普通 `load/store`	帧缓冲 `0x8000F000`	可用地址翻译保护；可用普通指令

19.3 数据传输：PIO vs DMA vs 中断/轮询

机制	谁搬数据	优点	缺点
Programmed I/O（PIO）	CPU 逐字节/逐字	硬件简单	占用 CPU 周期 ∝ 数据量
DMA	控制器直接搬	CPU 解放	需要一致性/缓存刷写；设置开销
中断	设备完成后发中断	适合不可预测事件	中断开销较高
轮询（polling）	CPU 定期查状态	低延迟、低开销（高频时）	空转浪费

实践中**混合**：高带宽网卡“第一个包用中断，后续包用轮询直到队列空”（讲义原话）。

19.4 驱动结构：上半部 / 下半部



部分	运行上下文	能做什么	不能做什么
上半部	进程上下文（系统调用路径）	睡眠、分配内存、拷贝到用户	长时间关中断
下半部	中断上下文	快速取数据、唤醒	**睡眠**、长时间持锁

19.5 三种接口风格

风格	语义	代表
阻塞（blocking）	调用直到完成	普通 `read/write`
非阻塞（non-blocking）	立刻返回已传输量	`O_NONBLOCK`
异步（asynchronous）	提交后立即返回，完成时通知	POSIX AIO、io_uring

19.6 设备分类

块设备（磁盘/SSD，按 512B/4KB 块访问，接口 `open/read/write/seek`）、**字符设备**（终端/串口，字节流，`get/put`）、**网络设备**（以太网/Wi-Fi，包，socket）。

19.7 关键数据结构与不变量

- 驱动向内核注册 `struct file_operations` 一类的函数表；
- 每个请求有状态机：`queued → in-progress → done/error`；
- **不变量**：中断处理程序必须能在“设备寄存器状态随时变化”的情况下正确工作；共享的队列需要关中断或自旋锁保护。

19.8 经典陷阱

1. **在中断上下文睡眠**：会 panic 或死锁。

2. **忘记内存屏障/`volatile`**：编译器可能把 MMIO 读写优化掉或重排。
3. **中断丢失（lost wakeup）**：先检查完成标志再睡眠，中间来了中断 ⇒ 永久睡眠；正确做法是关中断下检查并入队。
4. **DMA 与 cache 不一致**（设备写内存后 CPU 可能读到旧 cache 行，需 invalidate/flush）；设备会失败（介质错误、超时），驱动必须能重试/上报。

第 20 章 讲义 19：存储设备、磁盘调度与 SSD

20.1 动机与机制：磁盘几何与延迟分解

```
盘片(platter) → 磁道(track，同心圆，外侧扇区更多) → 扇区(512B/4KB)
柱面(cylinder) = 所有盘面上同一半径的磁道集合；每面一个磁头
```

```
磁盘访问时间 = 排队 + 控制器 + 寻道 + 旋转 + 传输
```

讲义给的算例（5ms 寻道、7200RPM、50MB/s、4KB 块）：

场景	计算	结果	有效带宽
随机读	5 + 4 + 0.082 ms	9.08 ms	≈ 451 KB/s
同柱面随机	4 + 0.082 ms	4.08 ms	≈ 1.03 MB/s
同磁道顺序	0.082 ms	0.082 ms	≈ 50 MB/s

结论：顺序访问比随机访问快两个数量级 ⇒ 文件系统要做**块聚集、预取、日志结构化**。

20.2 磁盘调度算法

算法	规则	优点	缺点
FIFO	按到达	公平	寻道多
SSTF	最近的请求优先	寻道少	远端饥饿
SCAN（电梯）	单向扫到底再反向	吞吐好	中间磁道更受宠
C-SCAN	只单向扫，回程不服务	更公平	回程浪费

20.3 SSD：没有寻道，但有擦除

特性	说明
读单位	页（4KB）
写单位	页，但**只能写空页**
擦除单位	块（256KB 量级），且**很慢**（ms 级）
寿命	每块有限擦写次数（MLC 约 1–10K 次）
对策 1：FTL	逻辑块 → 物理页的映射表，OS 无感
对策 2：Copy-on-Write	改写时写新页、改映射，不原地覆盖
对策 3：GC	后台把有效页搬走、擦除整块、加入 free list
对策 4：磨损均衡	把擦写均匀分布到所有块

讲义给的定性结论：SSD **随机读极快**（无寻道/旋转），**随机写较差**（擦除 + GC），因此 FTL 的 GC 策略对性能影响巨大；顺序写在 SSD 上仍然最优。

20.4 性能度量与排队论

指标	定义	公式
延迟（Latency）	完成一个任务的时间	`L`
响应时间	发起请求到拿到响应	含排队
吞吐（Throughput）	单位时间任务数	`B = 1/L`（单服务器）
有效带宽	传输量 / 总时间	`EffBW(n) = n/(S + n/B)`
半功率点	有效带宽达到峰值一半的传输量	`n = S·B`

流水线与多服务器：`k` 级流水线的吞吐 ≈ `k/L`；`k` 个并行服务器的吞吐 ⩽ `k/L`。**排队效应**：利用率 `u → 1` 时排队延迟趋于无穷，讲义近似 `Tq = Tser × (1/2)(1+C) × u/(1−u)`（`C` 为服务时间变异系数）——这就是“为什么 70% 利用率是很多系统的舒适区”。

20.5 关键数据结构与不变量

- 块设备接口：`block_read(blockno, buf)` / `block_write(...)`；维护 `read_cnt/write_cnt`（PintOS 有，Project 3 的测试要用）。
- 请求队列：按扇区号排序（电梯）；合并相邻请求。

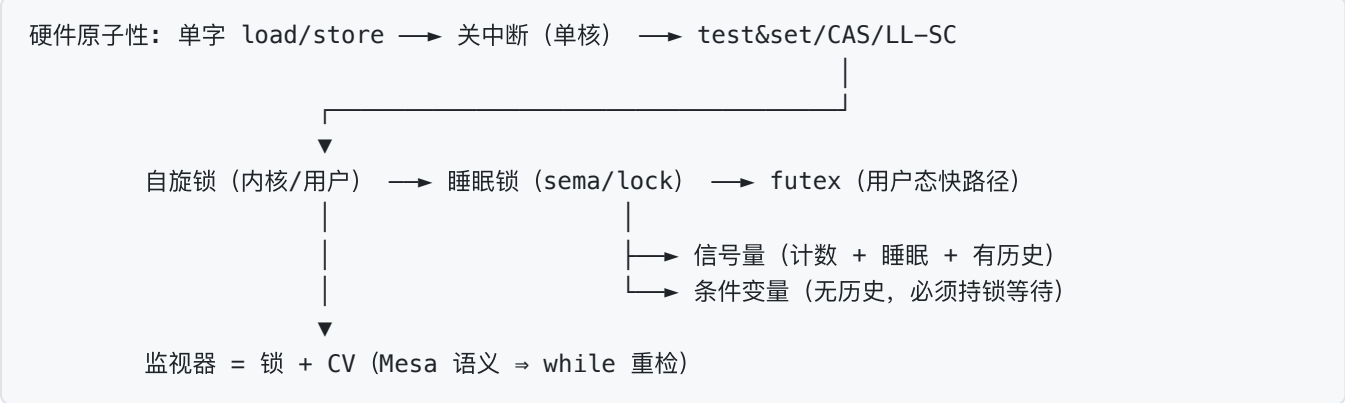
- FTL 映射表：逻辑页 → 物理页；必须持久化（或可重建）。

20.6 经典陷阱

1. **用平均寻道时间估算一切**：实际分布与队列深度强相关。
2. **忽略“写放大”**：SSD 上一个小改写可能触发整块 GC。
3. **把 SSD 当“更快的磁盘”而不改文件系统布局**：日志/对齐/TRIM 仍很重要。
4. **在 100% 利用率下做容量规划**（排队延迟爆炸）；忘记 `fsync`/flush 会让“已写”数据因设备缓存而丢失。

第 21 章 并发专题：一张图串起锁、信号量、监视器

21.1 全景



21.2 选择表（考场上最快的一张）

需求	用什么	理由
保护一小段共享数据	锁 / mutex	语义最简单
等待“缓冲区非空/非满”	CV（配锁）或信号量	CV 更清晰，sema 更轻
计数型资源（N 个连接槽）	信号量（初值 N）	天然计数
事件通知（“做完告诉我”）	信号量（初值 0）	有历史，不会丢
读多写少	读写锁	并发读
用户态高频无竞争	futex/三态锁	零系统调用

21.3 死锁检查清单

1. 所有锁是否有**全局顺序**?
2. 是否在持锁时调用可能加锁的函数（回调、`printf`、`malloc`、`file_write`）?
3. 是否在持锁时做阻塞 I/O?
4. `P` 的嵌套顺序是否与资源依赖一致?
5. 是否有可能“持锁睡眠、被唤醒后条件已变”?
6. 是否有“锁 + CV”之外的隐式锁（例如 PintOS 的 `filesys_lock`）?

21.4 并发 bug 的四个典型形态

形态	现象	检测
数据竞争	结果随调度变化	ThreadSanitizer、压力测试
原子性破坏	中间状态被看到	缩小临界区后重现 ⇒ 边界错
死锁/活锁	卡住或空转	`btthreadall`、超时

第 22 章 调度专题：策略选择的决策树

```
需要可预测的最坏延迟?
├── 是 → 硬实时?  → 是 → RMS/EDF（做可调度性测试）
│                  └ 否 → 优先级 + 带宽预留（CBS）
└── 否 → 交互式/通用?
          ├── 任务长度未知、I/O 混合 → MLFQ / CFS
          ├── 需要按权重分配 → lottery / stride / CFS
          └── 批处理、追求吞吐 → FCFS + 大时间片（或 SJF 近似）
```

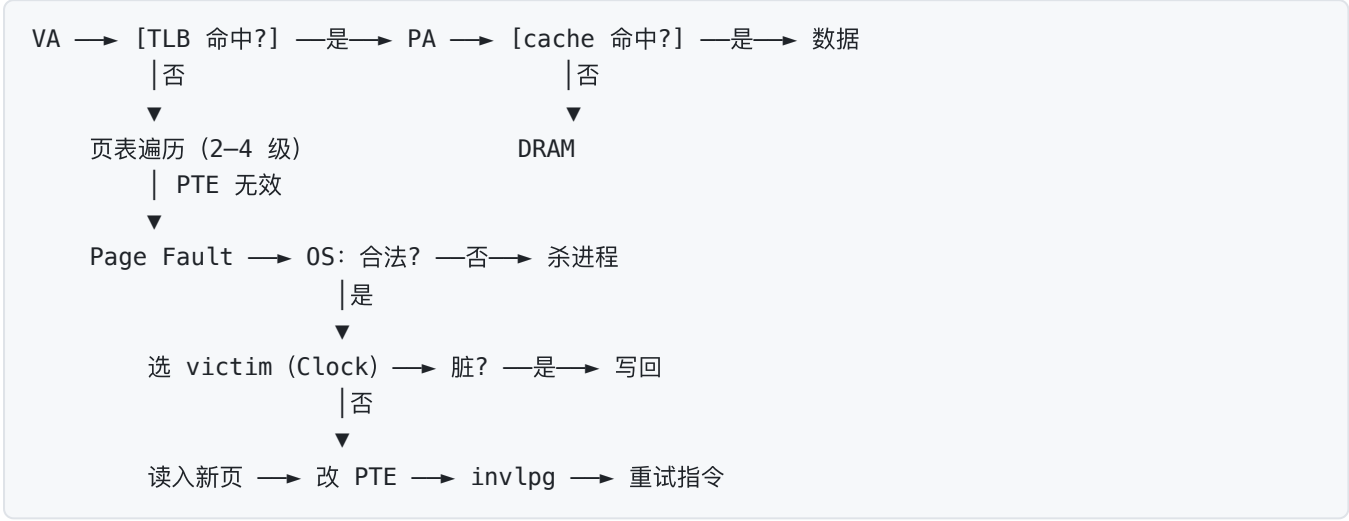
场景	推荐	关键参数	风险
桌面交互	CFS/MLFQ	target latency、min granularity	交互判定被欺骗
服务器吞吐	CFS + 绑核	权重、NUMA 亲和	尾延迟
实时控制	EDF/RMS	周期、WCET	过载崩溃
教学内核（PintOS）	严格优先级 + RR 同级	priority 0—63	饥饿、捐赠遗漏

三张必须记住的公式:

- RR 最大等待 `(n-1)q`;
- CFS 时间片 `Q_i = TargetLatency × w_i / Σw`，且 `Q_i ≥ MinGranularity`;

- RMS 判据 $\Sigma U_i \le n(2^{1/n}-1)$；EDF 判据 $\Sigma U_i \le 1$。

第 23 章 内存专题：从地址到页框的完整链路



层次	缓存什么	命中时间	缺失代价
寄存器/ALU	—	1 cycle	—
L1	数据/指令	~4 cycles	~12 cycles
TLB	翻译	~1 cycle	10–100 cycles（页表遍历）
L2/L3	数据	10–40 cycles	~200 cycles
DRAM	页	~200 cycles	—
磁盘/SSD	页	—	10μs–10ms

必背数字：4KB 页 ⇒ 12 位 offset；32 位两级页表 10–10–12；EAT 公式；缺页率需 < 2.5×10^−6 才能把减速控制在 10% 以内。

第 24 章 文件系统专题（讲义 20/22/23）

> **材料说明**：本学期站点上 Lecture 20、22、23 的 PDF **返回 404**，因此本章依据主页日程表的讲题（“File Systems 1: Performance, Queueing Theory, Filesystem Design”“Filesystem Design (Con't), Case Studies”“Buffer Cache, Reliability, Transactions”）、教材 A&D Ch 11/13/14 的公开目录，以及 OSTEP 对应章节（文件与目录、文件系统实现、FFS、崩溃一致性、日志结构文件系统）撰写。凡属推断均已标注。

24.1 文件系统的四层结构（OSTEP/A&D 通用模型）

```
应用 open/read/write/close → VFS（inode/dentry/file 统一接口）
  → 具体 FS（FFS/ext4：目录、inode、块分配、日志）
  → 块层（buffer cache、I/O 调度、RAID）→ 设备驱动 → 磁盘/SSD
```

24.2 inode 与目录

结构	内容	不变量
inode	类型、权限、大小、时间戳、块指针（直接/间接/双重间接）	每个 inode 有唯一 inode 号；链接计数 = 目录项数 + 打开数
目录	`(名字, inode 号)` 列表	名字在同一目录内唯一；`.` 与 `..` 必须存在
超级块	FS 元信息（块大小、inode 数、free map 位置）	挂载时校验

PintOS 的 inode 是“单扇区”结构（Project 3 要求改成索引结构，支持 8MiB 文件），这与 FFS 的“直接 12 + 一级间接 + 二级间接 + 三级间接”是同一思想。

24.3 空闲空间管理

方法	优点	缺点
位图（bitmap）	紧凑、易找连续块	需要扫描
空闲链表	分配快	碎片、遍历慢
分组链接	兼顾	实现复杂

24.4 缓冲缓存（buffer cache）

- 缓存**磁盘块**（不是文件页）；PintOS Project 3 要求 **64 块上限**、**写回（write-back）**、替换策略至少和 Clock 一样好；
- 必须处理：**同一块的并发访问**（正在读入的块不能被别人再读一遍）、**驱逐时的占用**（不能驱逐正在使用的块）；
- 元数据（inode、位图）比数据更值得缓存（讲义建议“account for the generally greater value of metadata”）。

24.5 崩溃一致性：FSCK vs 日志

方案	做法	代价
FSCK（事后扫描）	重启后全盘检查、修复	慢；可能无法恢复语义
预写日志（WAL/journaling）	先写日志（含校验和），再写原位；崩溃后重放	写放大；需要事务边界
日志结构 FS（LFS）	所有写都是追加，顺序写	需要 cleaner；读放大
软更新（soft updates）	按依赖顺序写，避免不一致窗口	规则复杂

WAL 的三条规则（OSTEP 的总结）：① 日志写完成再写 checkpoint；② checkpoint 完成才释放日志空间；③ 重放必须幂等（用校验和 + 事务 ID）。

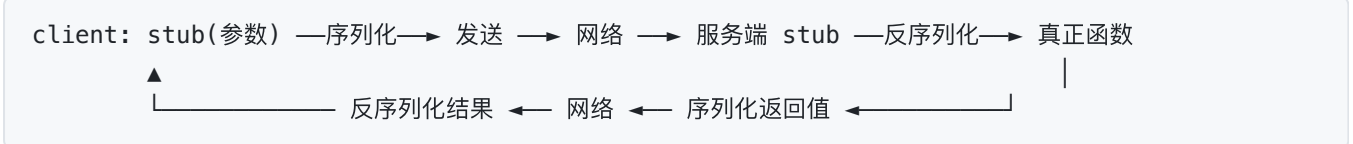
第 25 章 分布式专题（讲义 24–27）

> **材料说明**：Lecture 24–27 的 PDF 本学期站点返回 404；主页讲题依次为“Transactions, End-to-End Arguments, Distributed Decision Making”“Decision Making (Con't), Networking and TCP/IP”“Networking (Con't), RPC, Distributed File Systems NFS”“NFS, AFS, Chord”。本章按这些讲题 + 公开教材原理组织，涉及具体讲义细节处标注“推断”。配套材料：RPC Lab（`https://cs162.org/static/hw/lab-rpc/`）与 HW6 Raft。

25.1 为什么分布式难

难题	表现	课程对应
部分失败	消息丢失、节点假死	RPC 超时与幂等
不可靠网络	重复、乱序、延迟	事务 ID、去重
无全局时钟	无法定义“先后”	逻辑时钟、两阶段提交
并发	多副本同时修改	锁、租约、共识（Raft）

25.2 RPC：把调用变成消息



要点（RPC Lab 的描述与此一致）：**序列化/反序列化**、**接口定义**、**超时与重试**、**幂等性**（至少一次 vs 至多一次）、**失败语义**（服务器崩溃时客户端看到什么）。

25.3 事务与 ACID

属性	含义	实现手段
Atomicity	全做或全不做	日志 + undo/redo
Consistency	约束不被破坏	应用 + 数据库约束
Isolation	并发事务互不干扰	两阶段锁（2PL）、MVCC
Durability	提交后不丢	WAL + fsync

两阶段提交（2PC）：协调者 `prepare` → 各参与者投票 → 协调者 `commit/abort`。阻塞问题：协调者崩溃会让参与者长期持锁（推断：课程会讨论其阻塞性质与 3PC/Paxos 的动机）。

25.4 端到端论证（End-to-End Arguments）

核心思想（Saltzer/Reed/Clark）：**某个功能只有在网络边缘完整实现才可靠；在中间节点实现只能是优化。**例：校验和、重传、加密，端到端做才可信；中间做只能提高概率。

25.5 NFS / AFS / Chord

系统	设计目标	关键机制	代价
NFS	无状态、简单、跨平台	文件句柄（fh）、幂等操作、属性缓存	缓存一致性弱（靠超时/校验）
AFS	大规模、客户端缓存	整文件缓存 + 回调（callback）承诺	服务器需记录客户端状态
Chord	P2P 精确查找	一致性哈希 + finger table（O(log N) 跳）	维护环、处理加入/离开

```
Chord: m 位 ID 空间排成环；key 存于其后继节点；
finger[i] = 后继(n + 2^i) ⇒ 每跳至少前进剩余距离的一半 ⇒ O(log N)
```

一致性哈希的要点：节点增减只影响相邻区间（“最小迁移”），这是 P2P 与分布式缓存的基础。

第 26 章 PintOS 四个项目精讲

> 本章的命令、文件名、函数名均来自 PintOS 文档（`https://cs162.org/static/proj/pintos-docs/`）与四个项目页（`proj-pregame`、`proj-userprog`、`proj-threads`、`proj-filesys`）。评分比例与具体测试清单以当学期 autograder 为准。

26.1 Project 0：Pregame（个人，计入作业）

项	内容
目标	用 GDB 走通“BIOS → loader → kernel → 用户程序 → 崩溃 → 修复 → 系统调用”
关键文件	`threads/loader.S`、`threads/start.S`、`threads/init.c`、`userprog/process.c`、`userprog/syscall.c`、`lib/user/syscall.c`
任务	① 找出 `do-nothing` 崩溃的指令与地址；② 一行修改 `process.c` 让 `do-nothing` / `do-not-much` / `stack-align-v` 通过；③ 实现 `practice(i) = i+1`
交付	17 道书面题（Gradescope）+ 代码
常见 panic	用户栈未正确设置 ⇒ 访问 `argv` 时 page fault；`iret` 前 `if_` 结构错位

核心洞察：`do-nothing` 崩溃的根因是**用户栈上没有 `argc/argv`**，`main` 访问 `argv[0]` 时踩到未映射地址。修法是在 `start_process` 里按调用约定压入参数（Project 1 会正式实现）。

26.2 Project 1：User Programs

项	内容
目标	参数传递 + 15 个系统调用（6 个进程控制 + 9 个文件操作）+ 用户内存安全
关键文件	`userprog/process.c`（`process_execute/process_wait/load/setup_thread`）、`userprog/syscall.c`、`userprog/exception.c`（`page_fault`）、`threads/thread.h`（加字段）
必须实现	`practice, halt, exit, exec, wait, fork`；`create, remove, open, filesize, read, write, seek, tell, close`
实现顺序	参数传递 → `write`（否则测试全哑）→ `exit` → `exec/wait` → 文件系统调用 → `fork` → 拒绝写可执行文件
评分点	autograder 测试通过率 + 设计文档 + design review + 代码质量 + 报告 + 口试；需自写 2 个新测试
常见 panic	`ASSERT(is_thread(next))`、`page fault in kernel mode`（未验证用户指针）、`filesys` 并发导致 `sec_no < capacity` 断言

三个必须想清楚的设计点：

1. **父子同步**：`exec` 必须等子进程 `load` 成功/失败后才能返回（用信号量），否则 `exec` 的返回值语义不成立。
2. **`wait` 与资源释放**：子进程可能先于父进程退出；父进程可能永不 `wait`。PintOS 要求**两种情况都不泄漏**（典型做法：子进程退出时把 `struct thread` 的释放交给“最后一个引用者”）。
3. **用户指针检查**：逐页验证（`pagedir_get_page` + `is_user_vaddr`），注意 4 字节参数**跨页**的情况。

26.3 Project 2：Threads

项	内容
目标	① 高效 `timer_sleep`；② 严格优先级 + 优先级捐赠；③ 用户级线程（pthread）+ 用户级锁/信号量
关键文件	`devices/timer.c`、`threads/thread.c`（`thread_set_priority/thread_get_priority/thread_tick`）、`threads/synch.c`、`userprog/process.c`、`userprog/syscall.c`、`lib/user/pthread.c`
必做	`timer_sleep` 用 `thread_block` + 唤醒队列；`sema_down/sema_up`、`lock_acquire/lock_release`、`cond_wait/cond_signal` 按**有效优先级**选择等待者；`lock` 支持多源、嵌套、释放时回退的捐赠
用户级	`pthread_create/exit/join`、`get_tid`、`lock_init/acquire/release`、`sema_init/down/up`（允许每次系统调用陷入内核）
评分点	测试通过 + 设计文档 + 评审 + 报告；需自写 1 个新测试
常见 panic	`intr_context()` 断言（在中断里睡眠）、`list_empty` 误用、捐赠未回退导致优先级泄漏

优先级捐赠的三个难点（项目页明确列出）：多源捐赠（取最大）、释放时回退（恢复到 base 或剩余捐赠的最大值）、嵌套捐赠（H→M→L 时把 H 的优先级传给 L）。

26.4 Project 3：File Systems

项	内容
目标	① 64 块写回 buffer cache（替换策略 ⩾ Clock）；② 索引 inode 支持 8MiB 文件与增长；③ 子目录与相对/绝对路径；④ 细粒度同步（不得全局大锁）
关键文件	`filesys/cache.c`（新建）、`filesys/inode.c`、`filesys/directory.c`、`filesys/filesys.c`、`userprog/syscall.c`
新系统调用	`chdir, mkdir, readdir, isdir, inumber`
实现顺序（项目页建议）	buffer cache → 可扩展文件 → 子目录（先写路径解析）

项	内容
评分点	测试 + 设计文档 + checkpoint（不计分但要过）+ 报告；需自写 2 个 buffer cache 测试
常见 panic	`sec_no < d->capacity`（inode 已释放）、缓存块未标记“正在使用”导致重复读、持全局锁做 I/O 导致超时

同步要求的关键句：可以有一把保护 buffer cache 的全局锁，但**不能在持锁时做阻塞 I/O**；不同扇区的操作应能并发。

第 27 章 与已有笔记的衔接

已有笔记	本课程对应	建议读法
`CS111_操作系统原理学习笔记.md`	讲义 6–17 几乎一一对应（并发、锁、调度、VM、置换）	先读 CS111 建立原理，再用 CS162 的 PintOS 项目验证“真实内核里怎么写”
`CSAPP_深入理解计算机系统_学习笔记.md`	讲义 2/3/14–16 的硬件侧（ISA、cache、链接、异常控制流）	CSAPP 第 8 章（异常控制流）与第 9 章（虚拟内存）是 CS162 的先修直觉
`Stanford_CS144_计算机网络_学习笔记.md`	讲义 5–6（socket、TCP 5-tuple）与讲义 25–26（TCP/IP、RPC）	先有 CS144 的 TCP 状态机与拥塞控制，才看得懂“为什么 socket 是队列”
`UIUC_CS425_分布式系统_学习笔记.md` / `CS425_分布式系统_实战速查手册.md`	讲义 24–27（事务、2PC、NFS/AFS、Chord）	CS162 只给“一讲概览”，分布式细节以 CS425 为主
`CS144_计算机网络_项目上手速查.md`	对照它的“依赖图 + 测试读法 + 坑清单”写法	本文档第 26 章与配套速查同构，可对照阅读

差异提醒：CS111 用“用户态模拟 OS”（线程库、FUSE），CS162 用“真内核 + autograder”；CSAPP 的 malloc lab 与 CS162 的 HW4 目标相似但环境不同（前者用户态、后者 PintOS 用户程序）；不要把这几个 lab 的接口混为一谈。

第 28 章 复习清单与自测题

28.1 一页纸清单（考前 30 分钟）

- [ ] 四大抽象（Thread / Address Space / Process / Dual-mode）与“硬件→软件”映射；三种陷入内核（syscall/interrupt/trap）与 `iret`。

- [ ] 线程四态与 TCB 字段；`elem` 只能挂一个队列；`magic` 检测栈溢出。
- [ ] 锁实现阶梯：关中断 → test&set → 睡眠队列 → futex 三态；每步的局限。
- [ ] 信号量三用法（互斥/顺序/计数）；P 顺序错会死锁。
- [ ] 监视器：Mesa 语义、`while` 重检、`broadcast` 何时必须。
- [ ] 调度：FCFS/RR/SJF/SRTF/MLFQ/CFS 的优缺点与公式；优先级反转与三种解法。
- [ ] 实时：RMS 充分条件 `n(2^{1/n}-1)`、EDF 充要条件 `ΣU ≤ 1`。
- [ ] 死锁四条件与银行家算法；全局锁顺序。
- [ ] 地址翻译：B&B → 分段 → 分页；10–10–12；PTE 位；TLB 一致性。
- [ ] 请求调页：EAT 公式、缺页处理五步、COW、mmap。
- [ ] 置换（FIFO/MIN/LRU/Clock/Nth/SC list、Bélády 异常、工作集与抖动）；I/O（MMIO vs PIO、DMA、中断 vs 轮询、上/下半部）；磁盘（寻道+旋转+传输、SCAN/C-SCAN、SSD 的 FTL/GC/磨损均衡）。
- [ ] 文件系统：inode、目录、位图、buffer cache、WAL 三规则。
- [ ] 分布式：RPC 语义、ACID、2PC、端到端论证、NFS vs AFS、Chord 的 O(log N)。

28.2 自测题（答案紧随）

Q1 为什么“关中断实现锁”不能给用户态使用？ **A** 用户可关中断后死循环，定时器与 I/O 全停；且多核上关一个核无效。

Q2 三态锁（UNLOCKED/LOCKED/CONTESTED）相比两态锁省掉了什么？ **A** 无竞争时 release 不必进内核唤醒；只有 `CONTESTED` 才 `futex_wake`。

Q3 有界缓冲区中，为什么生产者必须先 `P(emptySlots)` 再 `P(mutex)`？ **A** 反序会在缓冲区满时持有 mutex 等空位，而消费者需要 mutex 才能取货 ⇒ 死锁。

Q4 Mesa 语义下为什么 `wait` 必须放在 `while` 里？ **A** signal 只把等待者放回就绪队列，锁可能被第三方抢走并改变条件；被唤醒后条件未必成立。

Q5 CFS 中 nice 值相差 5 的两个任务，CPU 份额比约多少？32 位、4KB 页、两级页表，8MB 连续内存的进程最少几页页表页？ **A** `1.25^5 ≈ 3.05` ⇒ 约 3:1（`weight = 1024/1.25^nice`）；8MB = 2048 页 = 2 个二级表 + 1 个页目录 = **3 页**。

Q7 缺页率 1/1000、内存访问 200ns、缺页服务 8ms，EAT 是多少？要控制 10% slowdown 需要多低的缺页率？ **A** `200ns + 0.001×8ms = 8.2µs`；需 `p < 2.5×10^-6`。

Q8 为什么 FIFO 会有 Bélády 异常，而 LRU 不会？ **A** LRU 是栈算法：M 框的驻留集一定是 M+1 框驻留集的子集；FIFO 不满足该性质。

Q9 优先级捐赠要处理哪三种情况？ **A** 多源（取最大）、释放时回退（base 或剩余最大值）、嵌套（H→M→L 传给 L）。

Q10 为什么 Project 3 允许全局 buffer cache 锁，却不允许“持锁做 I/O”？ **A** 持锁 I/O 会让不同文件的独立操作串行化，违背“不同扇区可并发”的要求，且可能超时。

第 29 章 参考来源

> 全部 URL 于 2026-11 访问。讲义 PDF 用 `pdftotext -layout` 提取文本后阅读；Lecture 20–28 的 PDF 在本学期站点返回 404（已实测），故第 24–25 章依据主页讲题与公开教材撰写。

29.1 课程与政策

内容	URL
课程主页（Fall 2026，含 Schedule / 项目与作业日期）	https://cs162.org/
政策页（先修、评分、考试、项目组队、LLM 政策）	https://cs162.org/policies/
资源页（教材、C 语言复习、历年考题）	https://cs162.org/resources/
PintOS 文档总入口	https://cs162.org/static/proj/pintos-docs/

29.2 每讲讲义（Lecture 1–19）

讲	主题	URL
1	What is an Operating System?	https://cs162.org/static/lectures/1.pdf
2	Four Fundamental OS Concepts	https://cs162.org/static/lectures/2.pdf
3	Abstractions 1: Threads and Processes	https://cs162.org/static/lectures/3.pdf
4	Abstractions 2: Process Management, Files and I/O	https://cs162.org/static/lectures/4.pdf
5	Abstractions 3: Files and I/O, Sockets, IPC	https://cs162.org/static/lectures/5.pdf
6	Abstractions 4 + Synchronization 1	https://cs162.org/static/lectures/6.pdf
7	Synchronization 1: Concurrency and Mutual Exclusion	https://cs162.org/static/lectures/7.pdf
8	Synchronization 2: Lock Impl., Atomics, Futex	https://cs162.org/static/lectures/8.pdf
9	Synchronization 3: Semaphores, Monitors, RW	https://cs162.org/static/lectures/9.pdf
10	Scheduling 1: Concepts and Classic Policies	https://cs162.org/static/lectures/10.pdf
11	Scheduling 2: Classic Policies, Case Studies	https://cs162.org/static/lectures/11.pdf
12	Scheduling 3: Starvation, Priority Inversion, RT	https://cs162.org/static/lectures/12.pdf
13	Scheduling 4: Realtime, Deadlock	https://cs162.org/static/lectures/13.pdf

讲	主题	URL
14	Memory 1: VM, Segments, Page Tables	https://cs162.org/static/lectures/14.pdf
15	Memory 2: Multi-Level Page Tables, TLBs	https://cs162.org/static/lectures/15.pdf
16	Memory 3: TLBs, Demand Paging	https://cs162.org/static/lectures/16.pdf
17	Memory 4: Demand Paging Policies	https://cs162.org/static/lectures/17.pdf
18	General I/O	https://cs162.org/static/lectures/18.pdf
19	Device Drivers, Storage Devices, Performance	https://cs162.org/static/lectures/19.pdf

29.3 项目与文档

内容	URL
Project 0 Pregame · Project 1 User Programs（含 Tasks/FAQ/Grading 子页）	https://cs162.org/static/proj/proj-pregame/ · https://cs162.org/static/proj/proj-userprog/
Project 2 Threads（含 alarm clock / priority / pthread 子页）	https://cs162.org/static/proj/proj-threads/
Project 3 File Systems（含 buffer cache / extensible files / subdirectories）	https://cs162.org/static/proj/proj-filesys/
PintOS 线程/`struct thread`、用户程序启动、用户内存访问	https://cs162.org/static/proj/pintos-docs/docs/threads/thread-struct/ · `.../userprog/program-startup/` · `.../userprog/accessing-user-mem/`
PintOS GDB / backtrace / panic 调试	https://cs162.org/static/proj/pintos-docs/docs/development/gdb/ · https://cs162.org/static/proj/pintos-docs/docs/development/backtraces/ · https://cs162.org/static/proj/pintos-docs/docs/overview/debugging-panics/
作业页（HW0–HW6、RPC Lab）	https://cs162.org/static/hw/hw-intro/ · `hw-list/` · `hw-shell/` · `hw-http/` · `hw-memory/` · `lab-rpc/`（同前缀）

29.4 教材与补充

内容	URL
A&D《Operating Systems: Principles and Practice》2nd ed. 官方站点	http://ospp.cs.washington.edu/
OSTEP 免费全文（章节号以 v1.10 在线目录为准）	https://pages.cs.wisc.edu/~remzi/OSTEP/

内容	URL
OSTEP 相关章：进程/受限直接执行、调度、MLFQ、比例份额、地址翻译、分页、TLB、交换、锁、CV、信号量、I/O 设备、磁盘、FS 实现、FFS、崩溃一致性、LFS、Flash、分布式系统、NFS、AFS	同上（按章名检索）
C 语言复习（课程 Ladder）	https://cs162.org/ladder/
RPC Lab	https://cs162.org/static/hw/lab-rpc/

29.5 未获取/受限材料清单

材料	状态	影响
Lecture 20、21、22、23 PDF	HTTP 404（本学期站点无）	第 24 章依据讲题 + 教材原理撰写
Lecture 24–28 PDF	HTTP 404	第 25 章依据讲题 + RPC Lab + 公开教材撰写，细节标注“推断”
评分细则/测试清单的精确权重	未公开	文中只写政策页给出的 36/36/18/10，其余标“以当学期主页为准”